

3DGEER: 3D GAUSSIAN RENDERING MADE EXACT AND EFFICIENT FOR GENERIC CAMERAS

Anonymous authors
Anonymous submission

ABSTRACT

3D Gaussian Splatting (3DGS) (Kerbl et al., 2023) achieves a compelling balance between rendering quality and efficiency, but that efficiency is purchased through a local affine approximation that replaces each 3D Gaussian with a 2D projection—an assumption whose error grows systematically under generic, large field-of-view (FoV) cameras and, most acutely, under the fisheye optics common to robotics and driving. Recent extensions relax individual facets of this approximation, yet none reconcile *exactness in projective geometry* with *real-time efficiency* across general camera models. We introduce 3DGEER, a Gaussian rendering framework that resolves this tension: it eliminates the projective approximation in both transmittance and ray-particle association while sustaining splatting-level throughput. Beginning from first principles, we derive a closed-form, differentiable expression for the transmittance integral of Gaussian density along a ray by mapping each anisotropic Gaussian into a canonical isotropic frame; the resulting formulation renders and optimizes Gaussian parameters exactly—free of projective linearization—under arbitrary camera models. To retain efficiency, we contribute two complementary components: the *Particle Bounding Frustum (PBF)*, an analytic, tightly bounding structure that resolves ray-Gaussian association at the frustum level without BVH traversal, and the *Bipolar Equiangular Projection (BEAP)*, a unified representation across FoV models that both accelerates association and supplies more uniform ray supervision. Across pinhole and fisheye benchmarks—ScanNet++, ZipNeRF, MipNeRF360, and egocentric Aria data—3DGEER consistently outperforms prior and concurrent methods on the reported metrics, runs $5\times$ faster than existing projective-exact ray-based baselines, and generalizes to wider FoVs unseen during training, establishing a new state of the art in real-time radiance field rendering. We are careful to delimit this claim: exactness here refers to the removal of projective approximation, not to a full physically exact volumetric model with analytic ordering and self-occlusion.

1 INTRODUCTION

Since its introduction, 3D Gaussian Splatting (3DGS) (Kerbl et al., 2023) has established volumetric particle rendering in the splatting formulation (Yu et al., 2024a; Yan et al., 2024; Lu et al., 2024; Kheradmand et al., 2024; Charatan et al., 2024; Mallick et al., 2024) as the dominant paradigm for neural rendering, prized for its rare combination of high visual fidelity and real-time speed. That speed, however, is inherited directly from EWA splatting (Zwicker et al., 2001), which applies a local affine approximation that projects each 3D Gaussian onto the image plane as a 2D Gaussian. Faithful in narrow field-of-view (FoV) regimes, this approximation degrades systematically under wide-FoV geometry: the nonlinear projection carries the true footprint far from any symmetric 2D Gaussian, and the resulting discrepancy is most severe in the fisheye imagery that pervades robotics and autonomous driving (Rashed et al., 2021; Courbon et al., 2007; Kumar et al., 2023; Zhang et al., 2016; Yogamani et al., 2019, 2024; Sekkat et al., 2022). Efforts to extend differentiable rendering to specific fisheye models (Liao et al., 2024) or to broader camera families (Huang et al., 2024b) mitigate particular symptoms, but each remains anchored to a 2D projective-space approximation and thus stops short of an exact, closed-form formulation.

A competing view maintains that exact rendering is unnecessary: Celarek et al. (2025) argue that scaling the Gaussian budget (up to 1M) suffices to absorb the projective approximation gap in

splatting. That evidence, however, is confined to small object-centric scenes and does not transfer to large-scale reconstruction, where the Gaussian count needed for adequate coverage fluctuates sharply across objects and frustums and renders indiscriminate scaling prohibitively expensive. We show, moreover, that under generic distorted cameras the projective error is substantially larger, so that increasing the Gaussian count alone cannot close the quality gap—a ceiling that persists no matter how aggressively the budget is scaled (see Fig. 5).

A parallel line of work confronts volumetric particle rendering directly, in a ray-tracing/marching formulation (Yu et al. 2024b; Condor et al. 2025; Mai et al. 2025; Moënné-Loccoz et al. 2024; Talegaonkar et al. 2025; Gu et al. 2025). Such formulations natively accommodate large-FoV camera models and are, by construction, free of projective approximation error. Their defining obstacle is efficiency—concretely, the cost of associating each rendering ray with the 3D particles that actually contribute to it. Substantial effort has been directed at this bottleneck, most notably the Bounding Volume Hierarchies (BVH) of EVER (Mai et al. 2025) and 3DGRT (Moënné-Loccoz et al. 2024); yet the algorithmic complexity and limited parallelizability of scene-level acceleration structures keep pure ray-based methods well short of 3DGS-level throughput. The hybrid 3DGUT (Wu et al. 2025) instead couples ray-traced rendering with rasterization-based association to recover speed, but in doing so it reinherits projective approximation error at the association stage—foregoing exactness in projective geometry and, in the limit, producing grid-line artifacts (see Fig. 1 Left). No prior method, in short, delivers both the geometric exactness of ray-based rendering and the efficiency of splatting; 3DGEER is the first to do so.

We resolve this dichotomy with 3DGEER, a 3D Gaussian Exact and Efficient Rendering method that attains *exactness in projective geometry* (defined precisely in Sec. A) while sustaining *efficiency comparable to splatting* for generic cameras, spanning both perspective and a range of fish-eye projection models. The scope of the exactness claim is deliberate and bounded: it denotes the elimination of projective linearization in transmittance and association, not a full physically exact volumetric model with analytic ordering and self-occlusion. Three tightly coupled contributions substantiate the claim: (i) We revisit the maximum-response heuristic for Gaussians (Keselman & Hebert 2022) from first principles and establish its exact connection to projective geometry; building on this analysis, we derive a differentiable rendering framework whose closed-form gradient chain is numerically stable by construction. (ii) To make ray-particle association efficient without conceding geometric exactness, we recast it as the matching of Camera Sub-Frustums (CSF) to Particle Bounding Frustums (PBF)—the frustum-level analogue of the tile-AABB mapping in 3DGS—and derive, for each 3D Gaussian, a closed-form solution for the tight PBF that encloses it, yielding highly efficient association from the true 3D covariance. (iii) We introduce the Bipolar Equiangular Projection (BEAP), a representation that samples rays uniformly in angular space for color supervision; this sampling simultaneously aligns image-space partitioning with the underlying CSF and improves reconstruction quality over conventional pinhole and equidistant projections.

Extensive experiments across fisheye and pinhole datasets—ScanNet++ (Yeshwanth et al. 2023), Zip-NeRF (Barron et al. 2023), Aria (Lv et al. 2025), and MipNeRF (Barron et al. 2021)—establish that 3DGEER consistently surpasses prior and concurrent methods, with PSNR gains of 0.9–4.8 dB under the challenging wide-FoV conditions it primarily targets and a further 0.3–2.0 dB on standard pinhole views. Its advantage is most decisive precisely where projective error is largest, yet it does not come at the expense of the narrow-FoV regime. Most tellingly, when trained only on narrow-FoV data, 3DGEER still outperforms splat-based methods when tested on wide-FoV cameras, evidencing a robust generalization to extremely wide-FoV rendering that follows directly from its projective exactness rather than from any privileged access to wide-FoV training data.

2 RELATED WORKS

2.1 SPLATTING-BASED GAUSSIAN RENDERING

Most 3DGS variants (Yu et al. 2024a; Zhang et al. 2024; Rota Bulò et al. 2024; Gao et al. 2025; Zhao et al. 2025; Kerbl et al. 2024; Hou et al. 2025; Liu et al. 2025) inherit the original EWA splatting strategy (Zwicker et al. 2001; Ren et al. 2002), projecting 3D Gaussians onto the image plane and modeling their footprints as 2D conics. To enable differentiable rendering under distorted, large-FoV camera models, methods such as FisheyeGS (Liao et al. 2024) and GS++ (Huang et al. 2024b) estimate the Jacobian of the equidistant or spherical projection and correct the 2D mean and

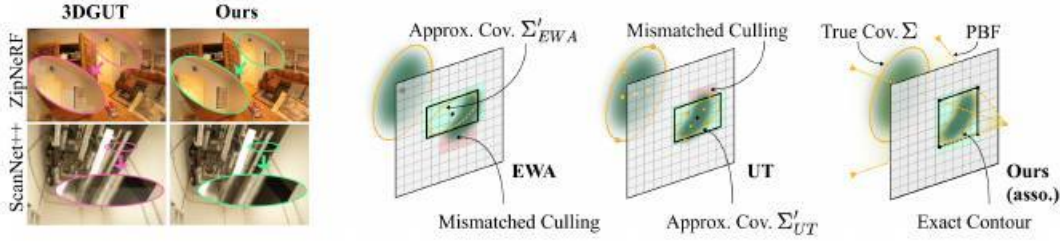


Figure 1: **Linear Approximation Error in Ray-Particle Association.** (Left) Grid-line artifacts induced by inaccurate UT-based association. (Right) A schematic contrasting our association scheme with prior alternatives: whereas EWA- and UT-based methods route the bound through an intermediate 2D conic approximation, our method computes the exact bounding structure directly from the true 3D covariance, dispensing with the projective intermediary altogether.

Table 1: **Summary of Particle Rendering Methods w.r.t. Exactness in Projective Geometry.** The top block collects splatting-based methods; the bottom block collects ray-based approaches, several of which nonetheless retain a projective-approximate association step (e.g., EWA or the Unscented Transform). Read across the columns—render type, particle-association method and scope, particle representation, large-FoV support, high FPS, and exactness—3DGEER is the only method that is simultaneously ray-based, exact, large-FoV capable, and high-FPS, and the only one that associates particles at the frustum level.

Method	Render	Particle Asso. Method	Particle Scope	Particle Type	Large FoV	High FPS	Exact Meth.
3DGS [Kerbl et al. 2023]	Splat.	EWA	Img.	3D Gauss.	✗	✓	✗
FisheyeGS [Liao et al. 2024]	Splat.	EWA	Img.	3D Gauss.	✓	✓	✗
GS++ [Huang et al. 2024b]	Splat.	EWA	Img.	3D Gauss.	✗	✓	✗
2DGS [Huang et al. 2024a]	Ray	Splat Bound.	Img.	2D Gauss.	✗	✓	✓
GOF [Yu et al. 2024b]	Ray	EWA	Img.	3D Gauss.	✗	✗	✗
EVER [Mai et al. 2025]	Ray	BVH	Scene	3D Ellip.	✓	✗	✓
3DGRT [Moënné-Loccoz et al. 2024]	Ray	BVH	Scene	3D Gauss.	✓	✗	✓
3DGUT [Wu et al. 2025]	Ray	UT	Img.	3D Gauss.	✓	✓	✗
HTGS [Hahlbohm et al. 2025]	Ray	Splat Bound.	Img.	3D Gauss.	✗	✓	✓
3DGEER (Ours)	Ray	PBF	Frust.	3D Gauss.	✓	✓	✓

covariance of each projected Gaussian accordingly. Yet these corrections still rest on a first-order Taylor expansion of a strongly non-linear projection: in wide-FoV regimes the neglected higher-order terms cease to be negligible, and the residual introduces substantial approximation error that surfaces as degraded reconstruction quality.

The root cause is geometric rather than incidental: under any non-linear camera model—ideal pin-hole or distorted alike—the true projection of a 3D Gaussian is not a symmetric 2D Gaussian. Any framework that approximates this inherently non-linear transformation with linear projective geometry therefore incurs an approximation error that no amount of refinement can fully remove. Our method sidesteps the limitation at its source, aggregating Gaussian density directly along rays within a volumetric rendering formulation and thereby dispensing with projective approximation entirely.

2.2 RAY-BASED GAUSSIAN RENDERING

Keselman & Hebert (2022) posit that a ray interacts with a Gaussian density field at the point of maximum intensity response. Building on this heuristic, 3DGRT (Moënné-Loccoz et al. 2024) constructs a 3D Gaussian ray-tracing framework that samples in ray space without explicit projection, thereby sidestepping wide-FoV distortion. Its accuracy, however, is bought at a steep computational cost: ray-particle association depends on scene-level BVH partitioning structures whose traversal is expensive, driving frame rates well below real time.

EVER (Mai et al. 2025) instead replaces Gaussians with constant-density ellipsoids, which admit a closed-form solution for transmittance-weighted ray integration. This substitution, however, curtails the representational capacity of full 3D Gaussians and, because it too relies on BVH structures, remains limited in runtime efficiency. 2DGS (Huang et al. 2024a) substitutes 2D surfels for 3D Gaussians, admitting exact ray-surface intersection at the cost of the full 3D representation. HTGS (Hahlbohm et al. 2025) restores the full Gaussian expression while retaining 2DGS’s

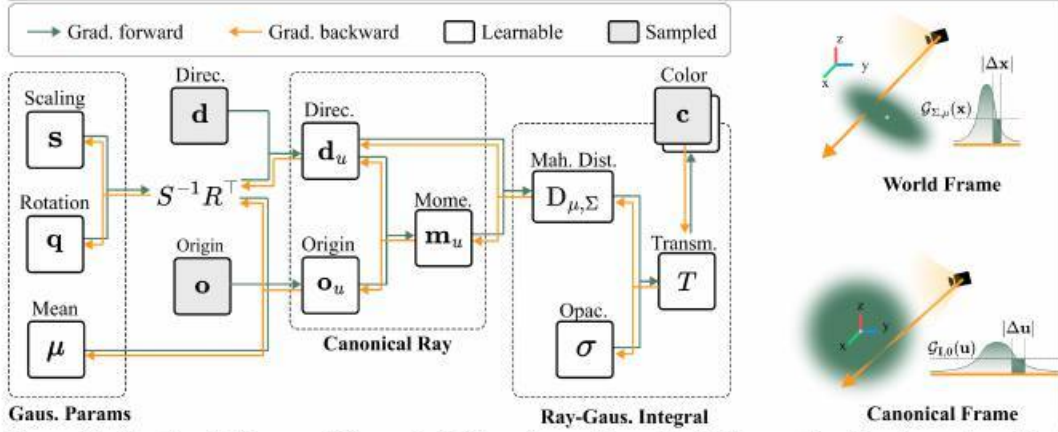


Figure 2: **Gradient Flow and Canonical Transformation.** (Left) Forward and backward gradient propagation coupling the transmittance to the Gaussian parameters s , q , μ and the opacity coefficient σ . (Right) Integration of 3D Gaussian density along a ray; the green box highlights the elementary transmittance—the product of density and ray-segment length—which the canonical change of variables preserves exactly (see Eq. B.5).

ray-splat association; because that association operates purely in screen space, however, it cannot guarantee exactness in projective geometry at wider FoVs. Our formulation, by contrast, derives closed-form bounds and a complete rendering pipeline directly in a camera-agnostic angular domain (Sec. D); we analyze in detail how our parametrization and rendering relate to, and computationally differ from, those of 2DGS and HTGS in Sec. D.3.

3DGUT (Wu et al. 2025), a follow-up to 3DGRT, recovers 3DGS-level frame rates by adopting splatting-based particle association. That acceleration, however, rests on the Unscented Transform (UT), a sampling-based approximation of the 2D conic in projective image space. GOF (Yu et al. 2024b) shares this limitation, applying the EWA construction directly to fit 2D bounding conics and projective centers for association. At extreme FoVs, the imprecise AABB estimates and mismatched tile-to-AABB associations they produce surface as visible grid-line artifacts (see Fig. 1 Left).

Distinct from all of the above, our approach solves ray-particle association directly at the frustum level in 3D space (see Fig. 1 Right), preserving the geometric exactness of ray-based rendering across arbitrary camera models while sustaining rendering speeds on par with 3DGS. Tab. 1 consolidates the representative methods that target projective approximation error and renders the decisive differences legible along the dimensions that matter—render type, association scope, large-FoV support, throughput, and exactness—situating 3DGEER as the sole entry that satisfies all of them at once.

3 METHOD

We present 3DGEER, a Gaussian rendering framework that jointly achieves geometric exactness and real-time efficiency under arbitrary-FoV cameras. Its design targets, in a single coherent construction, the two bottlenecks that have kept prior work from reconciling the two goals—*projective approximation* in splatting-based rendering and *BVH overhead* in ray-based rendering—through three complementary components:

- Ray-based rendering formulation (Sec. 3.1): a closed-form, differentiable solution for integrating Gaussian density along 3D rays that eliminates the linearization error intrinsic to projection-based splatting.
- Particle Bounding Frustum (PBF) (Sec. 3.2): a ray-particle association scheme that supplants BVH traversal with exact frustum-particle tests, delivering accuracy and efficiency within one construction rather than trading one for the other.
- Bipolar Equiangular Projection (BEAP) (Sec. 3.3): an image representation that unifies arbitrary-FoV camera models, accelerates PBF computation, and improves reconstruction quality.

The Appendix supplies the supporting technical development: (i) preliminaries and notation (Sec. B.1); (ii) the complete proof underlying Sec. 3.1 which establishes the equivalence between the maximum-response heuristic and exactness in projective geometry (Sec. B); (iii) the full backward-gradient derivations together with their numerical-stability analysis (Sec. C); and (iv) the complete derivation of geometrically exact association (Sec. D).

3.1 RENDERING WITH EXACT PROJECTIVE GEOMETRY

The central challenge in geometrically exact Gaussian rendering is to obtain a closed-form expression for integrating density and color along a ray. Our strategy is to map each anisotropic Gaussian in world space into a canonical coordinate system in which every Gaussian becomes isotropic. Under this mapping the transmittance integral reduces to a measure-preserving change of variables, which in turn admits a compact closed-form solution—exact rather than approximate, and independent of the camera model.

Canonical transformation. For a Gaussian with mean μ , rotation R , and scaling S , we define the mapping

$$\begin{bmatrix} \mathbf{x} \\ 1 \end{bmatrix} = \begin{bmatrix} RS & \mu \\ \mathbf{0} & 1 \end{bmatrix} \begin{bmatrix} \mathbf{u} \\ 1 \end{bmatrix}, \quad (1)$$

which carries a canonical coordinate $\mathbf{u}$ to its world coordinate $\mathbf{x}$. The mapping implies $\Delta\mathbf{x} = RS\Delta\mathbf{u}$ and absorbs the Jacobian determinant into the density measure (see Fig. 2 Right), so that every Gaussian collapses onto a single shared isotropic form

$$\mathcal{G}_{\mathbf{I},0}(\mathbf{u}) = \frac{1}{\rho} \exp\left(-\frac{1}{2}\|\mathbf{u}\|^2\right). \quad (2)$$

Closed-form transmittance. A world-space ray $\mathbf{r}(t) = \mathbf{o} + t\mathbf{d}$ transforms into its canonical counterpart $\mathbf{r}_u(t) = \mathbf{o}_u + t\mathbf{d}_u$, with

$$\mathbf{o}_u = S^{-1}R^\top(\mathbf{o} - \mu), \quad \mathbf{d}_u = S^{-1}R^\top\mathbf{d}. \quad (3)$$

The accumulated transmittance is then

$$T(\mathbf{o}, \mathbf{d}) = \sigma \int_{t \in \mathbb{R}} \mathcal{G}_{\mathbf{I},0}(\mathbf{o}_u + t\mathbf{d}_u) dt = \sigma \exp\left(-\frac{1}{2}D_{\mu,\Sigma}^2(\mathbf{o}, \mathbf{d})\right), \quad (4)$$

where

$$D_{\mu,\Sigma}^2(\mathbf{o}, \mathbf{d}) = \frac{\|\mathbf{o}_u \times \mathbf{d}_u\|^2}{\|\mathbf{d}_u\|^2} \quad (5)$$

is the squared perpendicular (Mahalanobis) distance from the ray to the Gaussian center.

Discussion. This result establishes that the transmittance is governed entirely by the ray’s distance to the Gaussian in canonical space, yielding a closed-form solution that is at once efficient and geometrically exact. The expression coincides with the “maximum-response” heuristic adopted in prior work (Keselman & Hebert, 2022; Moënné-Loccoz et al., 2024); our derivation, however, is what exposes the exactness in projective geometry that the heuristic had only tacitly assumed (see Sec. B). Because we carry the closed-form derivation through to the matching backward gradient chain (see Fig. 2 Left and Sec. C), 3DGEER is furthermore free of the numerical instabilities that beset prior ray-based work (Yu et al., 2024b), and it dispenses with the redundant post-processing—such as filtering out degenerate Gaussians—on which those methods depend.

3.2 RAY-PARTICLE ASSOCIATION WITH EXACT PROJECTIVE GEOMETRY

In any ray-based Gaussian renderer, both final accuracy—most acutely under large-FoV camera models—and runtime efficiency are governed by the ray-particle association strategy, a component that prior work has largely left unexamined. We reformulate association as

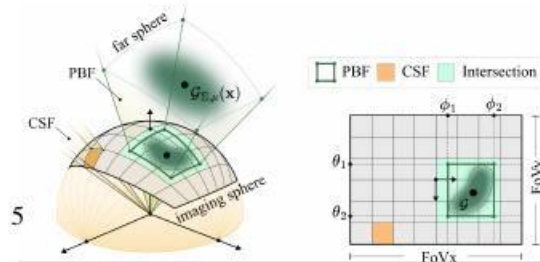


Figure 3: **PBF-CSF Association.** (Left) The PBF, delimited by four planes tangent to a 2D

a frustum-level matching problem: to identify the Gaussians that contribute to each ray, we map Camera Sub-Frustums (CSFs) to Particle Bounding Frustums (PBFs), the tight frustums that enclose each Gaussian at a chosen distance from its center (Fig. 3). A Gaussian is assigned to a CSF precisely when its PBF intersects that CSF, and every Gaussian so assigned is treated as a contributor to all rays within the CSF—a formulation that keeps the association exact while remaining amenable to massively parallel evaluation.

To compute a PBF efficiently, we first introduce two spherical angles (see Fig. E.1) in camera space:

$$\theta = \arctan\left(\frac{\mathbf{d}_{c,x}}{\mathbf{d}_{c,z}}\right), \quad \phi = \arctan\left(\frac{\mathbf{d}_{c,y}}{\mathbf{d}_{c,z}}\right), \quad (6)$$

where $(\mathbf{d}_{c,x}, \mathbf{d}_{c,y}, \mathbf{d}_{c,z})$ is the normalized direction vector toward the camera (view) space point $\mathbf{d}_c$. The angles θ and ϕ denote, respectively, the ray’s incidence angles in the horizontal (xz) and vertical (yz) planes, each confined to $[-\frac{\pi}{2}, \frac{\pi}{2}]$. A PBF is then delimited by four planes—two aligned with the θ angle and two with the ϕ angle (Fig. 3):

$$\mathbf{g}_{\theta_{1,2}} = (-1, 0, \tan \theta_{1,2}, 0)^\top, \quad \mathbf{g}_{\phi_{1,2}} = (0, -1, \tan \phi_{1,2}, 0)^\top, \quad (7)$$

where $\theta_1 < \theta < \theta_2$ and $\phi_1 < \phi < \phi_2$ fix the angular limits.

For a camera with extrinsic matrix $V = [R_c | \mathbf{t}_c] \in \text{SE}(3)$ and canonical-to-world transformation H (see Eq. B.4), we transport these view-space planes into canonical space by applying the inverse transpose of the camera-to-canonical matrix $(VH)^{-1}$, obtaining the constraints that define the PBF. Taking the bounds of θ as the representative case, the canonical plane reads:

$$\mathbf{g}_{u_{1,2}} = (VH)^\top \mathbf{g}_{\theta_{1,2}} = -(VH)_0 + \tan \theta_{1,2} (VH)_2, \quad (8)$$

where $(VH)_i$ denotes the i -th row of the matrix, written as a column vector.

For a Gaussian ellipsoid defined by a λ -standard-deviation contour, the angular bounds of the local frustum follow from the tangency condition between the transformed planes and the ellipsoidal contour, a condition that reduces to the quadratic equations:

$$((\lambda^2, \lambda^2, \lambda^2, -1) \circ \mathbf{g}_u^\top) \cdot \mathbf{g}_u = 0, \quad (9)$$

where $\circ$ denotes the Hadamard (element-wise) product and λ controls the spread of the contour, typically set to 3. Imposing the identical constraint on $\mathbf{g}_v^\top$ yields the corresponding angular bounds $\phi_{1,2}$. The roots of these quadratics supply the tangent values of the angular bounds $[\theta_1, \theta_2]$ and $[\phi_1, \phi_2]$ that fix the PBF angular limits. Crucially, because these quadratics are posed under canonical-space constraints, they operate directly on the true 3D covariance Σ_c and 3D mean μ_c of the Gaussian in camera space. Concretely, for the bounds of θ , the quantity $c = \tan \theta_{1,2}$ satisfies:

$$\mathcal{T}_{22}c^2 - 2\mathcal{T}_{02}c + \mathcal{T}_{00} = 0, \quad \text{where} \quad \mathcal{T}_{ij} = \lambda^2 \Sigma_c^{i,j} - (\mu_c \mu_c^\top)^{i,j}. \quad (10)$$

This formulation is inexpensive to solve (see Sec. D.1 for details) and, by operating directly on the true 3D covariance, obviates at once both the intermediate conic approximations of EWA and UT and the screen-space constraints that cap the usable FoV in HTGS and 2DGS. Exactness and efficiency thus emerge from the same construction rather than being traded against each other.

3.3 BIPOLAR EQUIANGULAR PROJECTION (BEAP)

Three complementary properties motivate the BEAP image representation. First, it represents images from large-FoV cameras without any FoV loss. Second, it endows image tiles and their corresponding CSFs with an identical parameterization—two θ and two ϕ angles—rendering PBF association highly efficient and mapping cleanly onto GPU parallelization. Third, the induced ray sampling is

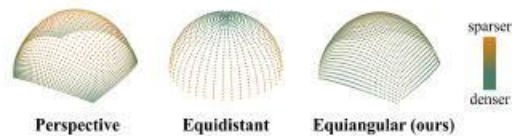


Figure 4: **Comparison of Ray Distributions under Varying Projections.** Projecting pixels from three imaging spaces onto the unit sphere exposes their induced ray distributions; of the three, our BEAP projection attains the most uniform coverage of the sphere.

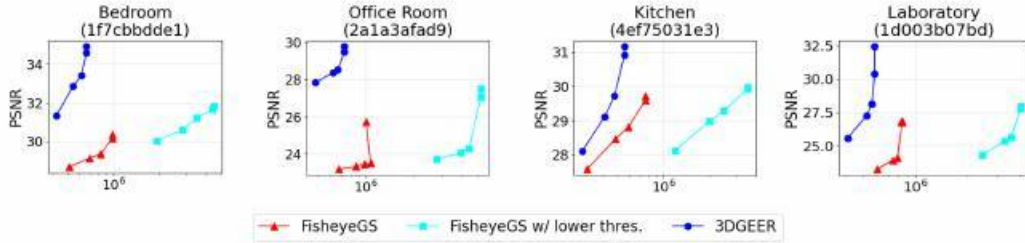


Figure 5: **PSNR↑ Trends When Scaling Up Gaussian Counts.** Brute-force scaling of the Gaussian budget does not close the gap in projective exactness: the splat-based curve saturates well below 3DGEER even at several-fold higher Gaussian counts, so that the deficit is one of formulation rather than capacity (full metrics in Fig. K.3 and Tab. K.3).

markedly more uniform in 3D space than that of conventional projective rendering, which we find empirically improves both training efficiency and reconstruction quality. The three properties are mutually reinforcing: the shared angular parameterization that makes association efficient is the same one that yields uniform sampling and full FoV coverage.

Concretely, we sample rays uniformly in the spherical angular coordinates (θ, ϕ) (see Eq. 6) and map them back to a discrete $w \times h$ image space through a linear transformation composed of a scaling factor and a center shift:

$$(x, y) = \left\lfloor \frac{w\theta}{\text{FoV}_x} + \frac{w+1}{2}, \frac{h\phi}{\text{FoV}_y} + \frac{h+1}{2} \right\rfloor, \quad (11)$$

where $\text{FoV}_x, \text{FoV}_y$ are the camera’s horizontal and vertical fields of view. As Fig. 4 illustrates, uniform sampling in (θ, ϕ) yields markedly more balanced spatial coverage of the view frustum. This behavior contrasts sharply with projective sampling, which over-allocates samples toward the periphery, and with equidistant projection, which conversely oversamples near the image center; BEAP alone distributes rays evenly across the sphere.

4 EXPERIMENTS

We evaluate 3DGEER across datasets that span both pinhole and fisheye camera models, so that the assessment covers the full spectrum from narrow to wide FoV rather than any single operating point. Unless otherwise stated, all experiments run on a single RTX 4090 GPU with 64GB of RAM as the default configuration; complete implementation details appear in Sec. G.

Datasets. To probe generic distorted cameras—the regime in which projective error is most consequential—we evaluate on all scenes of Fisheye ZipNeRF (Barron et al., 2023), five scenes of ScanNet++ (Yeshwanth et al., 2023), and two scenes of Aria (Lv et al., 2025). The ZipNeRF and ScanNet++ cameras span a 180° diagonal FoV, while Aria covers a 110° circular FoV. For the pinhole setting we use Pinhole ZipNeRF and MipNeRF360 (Barron et al., 2021). Taken together, these benchmarks exercise the method across the full distortion range it is designed to handle, from mild pinhole perspective to severe wide-FoV fisheye.

Evaluation Metrics and Protocol. We report reconstruction quality via PSNR↑, SSIM↑, and LPIPS↓, and efficiency via runtime (FPS↑) and Gaussian count↓. To eliminate representation bias across methods with differing internal spaces, every result is projected back to the native image space for full-FoV evaluation, and all methods are compared after a common budget of 30k training iterations. This protocol ensures that reported differences reflect the rendering formulation itself rather than mismatched evaluation spaces or training lengths.



Figure 6: **Extreme-FoV Generalization.** Under an identical extreme-FoV test condition, 3DGEER reconstructs complete scenes with fewer Gaussians, whereas FisheyeGS degrades severely toward the periphery.

Table 2: **Comparison on the ScanNet++ Dataset.** Each method is trained on either the full FoV or the central region alone, then evaluated on the full, central, and peripheral regions so as to isolate robustness under increasing distortion. Region-wise metrics are pixel-mask weighted; LPIPS is reported only at full FoV, since it is not cleanly separable by region. 3DGEER attains the best full-FoV PSNR/SSIM/LPIPS and the best peripheral scores; strikingly, even when trained on the central region alone it still surpasses FisheyeGS across every region, indicating that its advantage stems from projective exactness rather than from wider training data (Tab. K.2 reports full statistics).

Method	Training FoV	Training Space	Full FoV			Central		Peripheral	
			PSNR $\uparrow$	SSIM $\uparrow$	LPIPS $\downarrow$	PSNR $\uparrow$	SSIM $\uparrow$	PSNR $\uparrow$	SSIM $\uparrow$
3DGS	Central	Persp.	-	-	-	31.26	0.948	-	-
FisheyeGS	Full	Eq.	27.81	0.946	0.139	32.44	0.956	23.28	0.914
EVER	Full	KB	29.47	0.924	0.167	29.93	0.924	28.72	0.925
3DGUT	Full	Eq.	30.64	0.944	0.150	31.87	0.945	28.84	0.937
3DGEER (Ours)	Central	BEAP	29.93	0.949	0.130	32.86	0.954	26.64	0.930
3DGEER (Ours)	Full	BEAP	31.50	0.953	0.126	32.64	0.955	28.94	0.945

4.1 SCALING GAUSSIANS VS. PROJECTIVE EXACTNESS IN WIDE-FOV CAMERAS

Celarek et al. (2025) contend that merely scaling the Gaussian budget (up to 1M) suffices to absorb the linear-approximation gap between splat-based and ray-based methods. That claim, however, is established only on small object-centric scenes and leaves wide-FoV distortion unaddressed. To test it directly on the regime it omits, we scale the splat-based fisheye method FisheyeGS on ScanNet++ by lowering its densification threshold, driving the Gaussian count to 3.6–5M (on an H200).

The outcome is unambiguous (Fig. 5): at matched Gaussian counts, the red (FisheyeGS) and blue (3DGEER) curves remain separated by a persistent PSNR gap—the signature of a deficit in projective exactness—throughout the densification trajectory from 4k to 30k iterations, beyond which densification typically saturates. The light-blue curve makes the ceiling explicit: even after a substantial increase in memory and computation, scaled-up FisheyeGS saturates at 29.3 PSNR, whereas 3DGEER reaches 32.1 with only 550–700K Gaussians—roughly an order of magnitude fewer. Scaling the Gaussian budget, then, cannot substitute for projective exactness in the wide-FoV regime; the gap is structural, not a matter of capacity. Full statistics appear in Tab. K.3

4.2 COMPREHENSIVE EVALUATION ON ALL-FOV NOVEL VIEW SYNTHESIS

We benchmark 3DGEER against representative baselines on both fisheye and pinhole datasets, supplementing each comparison with analysis tailored to the properties of the dataset in question.

ScanNet++. 3DGEER outperforms every baseline on every metric (Tab. 2). To disentangle the effect of distortion, we evaluate the central and peripheral regions separately (protocol in Sec. G): the competing methods hold up at the center but deteriorate toward the periphery, whereas 3DGEER preserves high fidelity across the entire field, yielding sharper boundaries and fewer artifacts (Fig. K.4). Most tellingly, even when trained only on central crops, 3DGEER still surpasses FisheyeGS across

Table 3: **Comparison on the ZipNeRF Dataset.** Following a cross-camera generalization protocol, each method is trained on either Fisheye (FE, 1/8 resolution) or Pinhole (PH, 1/4 resolution) data and evaluated on both Fisheye and Pinhole test sets. 3DGEER leads on PSNR and SSIM across both training sources and both test cameras, and is the strongest overall precisely in the demanding pinhole-train/fisheye-test transfer—the setting that most stresses robustness to a change of camera model (Tab. K.9 reports full statistics).

Train Data Test Data Method	1/8 FE		1/8 FE		1/8 FE		1/4 PH		1/4 PH		1/4 PH	
	FE PH		FE PH		FE PH		FE PH		FE PH		FE PH	
	PSNR $\uparrow$		SSIM $\uparrow$		LPIPS $\downarrow$		PSNR $\uparrow$		SSIM $\uparrow$		LPIPS $\downarrow$	
FisheyeGS	23.18	26.44	0.858	0.868	0.211	0.239	19.43	26.61	0.791	0.889	0.247	0.212
EVER	25.17	26.14	0.880	0.851	0.153	0.237	22.90	26.45	0.835	0.862	0.207	0.222
3DGUT	24.77	25.59	0.879	0.804	0.183	0.324	18.61	25.96	0.662	0.841	0.300	0.270
3DGEER (Ours)	26.24	27.62	0.897	0.888	0.140	0.214	23.39	27.61	0.846	0.863	0.209	0.254

Table 5: **Training-Time Transmittance Replacement on ScanNet++.** Substituting a splatting-based transmittance inflates the Gaussian count—the optimizer densifies to compensate for the larger approximation error—yet a perceptual gap to our projective-exact formulation persists throughout, confirming that densification cannot recover the exactness that the substitution forfeits (Tab. K.7 reports full statistics).

Transmittance Method	Full FoV			Central		Peripheral		Avg. Gaussian Number (k) $\downarrow$
	PSNR $\uparrow$	SSIM $\uparrow$	LPIPS $\downarrow$	PSNR $\uparrow$	SSIM $\uparrow$	PSNR $\uparrow$	SSIM $\uparrow$	
3DGS Splats	22.86	0.878	0.177	27.16	0.884	18.56	0.855	1396.4
FisheyeGS Splats	27.90	0.948	0.141	32.49	0.957	23.36	0.917	920.6
Projective-Exact (Ours)	31.50	0.953	0.126	32.64	0.955	28.94	0.945	591.8

all regions—direct evidence that its advantage originates in projective exactness rather than in access to wider training data.

We further probe extreme-FoV generalization (Fig. 6) by training at the native dataset FoV (180° diagonal) and testing at a substantially wider FoV (180° circular). 3DGEER reconstructs complete scenes with fewer Gaussians, while splatting-based methods such as FisheyeGS degrade sharply at the periphery, where their projective approximation breaks down entirely (additional extreme-FoV visuals in Fig. K.1-K.2).

ZipNeRF. Because ZipNeRF supplies frame-wise aligned pinhole and fisheye views, we extend the standard fisheye evaluation with a *cross-camera generalization* protocol—training on fisheye and testing on pinhole, and vice versa—which stresses each method’s robustness to a change of camera model. 3DGEER leads on PSNR and SSIM across both training sources and both test cameras (Tab. 3), exploiting large-FoV training data to the full through geometrically exact rendering; trained on pinhole data, it still delivers clear PSNR gains on both test cameras. FisheyeGS, by contrast, is competitive only in the pinhole-to-pinhole case, and 3DGUT collapses when tested on fisheye after pinhole training. The pattern is consistent and diagnostic: only the geometrically exact methods—EVER and 3DGEER—generalize robustly across cameras, and among these 3DGEER surpasses EVER in every case. Qualitatively (Fig. 7), 3DGEER recovers finer detail and resists peripheral artifacts most visibly in the hardest pinhole-to-fisheye transfer.

Table 4: **Quantitative Results on the MipNeRF360 Dataset.** Alongside quality metrics, FPS is measured on an RTX 4090 as the default benchmark; figures taken from the original works are marked † (RTX 6000 Ada) and ‡ (RTX 5090). Even in this narrow-FoV regime, where the projective approximation is most benign, 3DGEER attains the best PSNR and LPIPS while running at splatting-comparable frame rates—an order of magnitude faster than the exact ray-tracing baselines EVER and 3DGRT.

Method	PSNR $\uparrow$	SSIM $\uparrow$	LPIPS $\downarrow$	FPS $\uparrow$
3DGS	27.21	0.815	0.214	343
EVER	27.51	0.825	0.233	36
3DGRT	27.20	0.818	0.248	52 † / 68 ‡
3DGUT	27.26	0.810	0.218	265 † / 317 ‡
3DGEER (Ours)	27.76	0.821	0.210	327

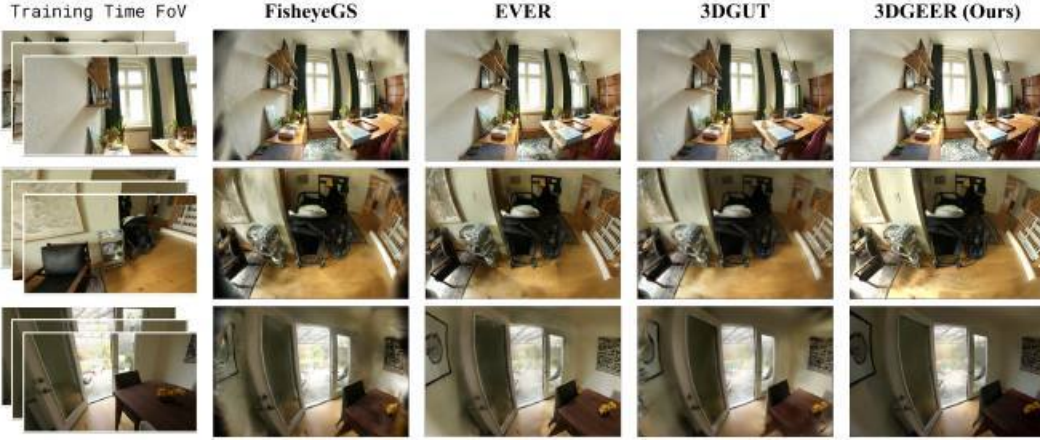


Figure 7: **Qualitative Results on the ZipNeRF Dataset with Cross-Camera Generalization.** The gap is most pronounced in the challenging Pinhole (PH) train–Fisheye (FE) test setting, where 3DGEER retains fine detail and suppresses the peripheral artifacts that the baselines cannot avoid.

We further evaluate 3DGEER on the **MipNeRF360** dataset under standard narrow-FoV conditions, establishing that its advantages are not confined to the wide-FoV regime for which it was designed. 3DGEER attains the best PSNR and LPIPS among all compared methods (Tab. 4): it runs over $5\times$ faster than the exact ray-marching/tracing methods EVER and 3DGRT—and is the only projective-exact method whose runtime is comparable to 3DGS—while at the same time surpassing the projective-approximation baselines 3DGS and 3DGUT on PSNR, SSIM, and LPIPS alike. It thereby reconciles exactness with speed even where the projective approximation is at its most benign, so that adopting exact rendering incurs no penalty in the narrow-FoV setting. Additional quantitative results on **Aria**—an egocentric fisheye dataset (moderate 110° circular FoV) captured under low light—together with further visuals appear in Sec. J.

4.3 COMPREHENSIVE RUNTIME ANALYSIS

We analyze runtime on the RTX 4090 against optimized splatting pipelines—3DGS on **MipNeRF360** and FisheyeGS on **ScanNet++**. Although ray-based rendering carries a slightly higher cost in the rendering stage proper, PBF’s efficient ray–particle association sharply curtails Gaussian duplication and sorting overhead (stage-wise timings in Tab. K.4), so that total efficiency remains on par with these highly optimized splatting pipelines—and, on **ScanNet++**, in fact exceeds FisheyeGS in total time.

The mechanism is direct. PBF produces very tight bounds, associating each tile with only ~ 475 Gaussians on average— $3\text{--}5\times$ fewer than EWA or UT (Tab. K.6)—which translates into the $2.5\text{--}5\times$ association-stage speedup reported in Tab. K.5. Projection-based schemes (EWA, UT), by conservatively overestimating screen-space extent, incur excessive overdraw and inefficient memory access; even when accelerated with SnugBox (Hanson et al. 2025) they remain limited by miscentered density approximations that frequently aggravate artifacts (Fig. I). Beyond speed, PBF preserves geometric accuracy and numerical stability by eliminating projective approximation error at the association stage: as Tab. K.6 shows, PBF rendering stays artifact-free even on Gaussian fields trained under the competing association schemes.

4.4 LOCAL AFFINE APPROXIMATION VS. PROJECTIVE-EXACT INTEGRAL

To isolate the contribution of our projective-exact transmittance formulation (Eq. 4), we substitute for it the widely used local affine splat (the 2D conic approximated in 3DGS and FisheyeGS), holding every other component of the pipeline fixed.

Rendering-Time Approximation. To sharpen the contrast between the two rendering formulations, we replace the transmittance with the splatting-based approximation at test time only, retaining every projective-exact component during training. When the splat approximation drives transmittance while association is still computed under exact projective geometry, the resulting inconsistency pro-

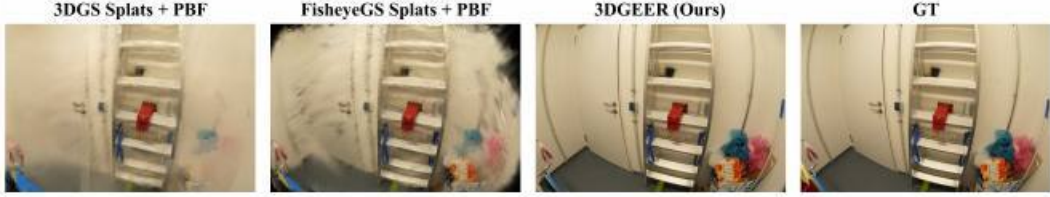


Figure 8: **Rendering-Time Transmittance Function Replacement.** Swapping the projective-exact transmittance for the splatting approximation at test time alone—while retaining exact association—produces mismatched culling, isolating the transmittance formulation itself as the source of the discrepancy.

Table 6: **BEAP vs. Alternative Supervision Schemes.** With the projective-exact integral and PBF association held fixed and only the supervision sampling varied, BEAP outperforms every alternative projection on all metrics on ScanNet++, isolating the ray-sampling distribution as the source of the gain (Tab. K.8 reports full statistics).

Method	Training FoV	Training Space	Full FoV			Central		Peripheral	
			PSNR $\uparrow$	SSIM $\uparrow$	LPIPS $\downarrow$	PSNR $\uparrow$	SSIM $\uparrow$	PSNR $\uparrow$	SSIM $\uparrow$
3DGEER w/o BEAP	Central	Persp.	29.84	0.943	0.131	32.21	0.951	26.23	0.915
3DGEER w/o BEAP	Full	Persp.	21.11	0.853	0.300	21.32	0.855	20.46	0.846
3DGEER w/o BEAP	Full	Eq.	31.05	0.948	0.135	32.21	0.952	28.56	0.935
3DGEER (Ours)	Full	BEAP	31.50	0.953	0.126	32.64	0.955	28.94	0.945

duces mismatched culling (Fig. 8). This controlled setting magnifies—and thereby explains—the grid-line artifact reported in 3DGUT (Fig. 1), where the same mismatch arises in the opposite direction, pairing ray-based transmittance with conic-approximate association.

Training-Time Approximation. On ScanNet++ (Tab. 5), when the splatting-based transmittance is used during training as well, with association held projective-exact, the optimizer and densification compensate only partially for the approximation error and the cross-view inconsistency it introduces. The larger loss elicits stronger gradients throughout training and, in consequence, a 1.6–2.4 $\times$ increase in the number of Gaussians needed to fit the scene. Even at this inflated budget the perceptual gap induced by the approximation persists, confirming once more that the error cannot be fully absorbed by densification alone.

4.5 BEAP ON/OFF STUDY

We compare BEAP supervision against other widely used projection models on ScanNet++. Holding the projective-exact integral and PBF association fixed, we vary only the ray-sampling distribution used for supervision—pinhole, equidistant, and BEAP—so that any difference is attributable to the sampling alone. Supervision is applied by interpolating the ground-truth color for each sampled ray (Sec. E), and evaluation is always performed in the native image space, so the comparison does not favor any supervision domain. BEAP supervision outperforms every alternative on all metrics (Tab. 6 Fig. K.5): its more uniform angular sampling supplies smoother, more consistent gradients and thereby drives better convergence on fine detail.

The gap widens under pinhole cameras, where—at fixed resolution—a well-balanced distribution of information proves more decisive than a wider FoV. Indeed, training in the perspective domain at full FoV can even underperform the central region alone (Tab. 6, rows 1–2), because large-FoV images (Fig. K.2 Left) exhibit highly non-uniform angular sampling: informative content concentrates at the center, while the less informative periphery, though it dominates the image area, is disproportionately oversampled. BEAP corrects this imbalance directly by equalizing angular coverage.

5 CONCLUSIONS

We introduced 3DGEER, a complete closed-form solution for projective-exact volumetric Gaussian rendering derived from first principles. Around this formulation we built two components that ren-

der exactness practical rather than merely theoretical: an efficient PBF construction that resolves ray-particle association at the frustum level, sustaining high-speed rendering without forfeiting exactness in projective geometry, and the BEAP image representation, which supplies effective, uniformly sampled color supervision under arbitrary-FoV camera models. Extensive experiments across pinhole and fisheye datasets establish that 3DGEER consistently outperforms state-of-the-art methods, with especially decisive gains in the challenging wide-FoV regime and no loss of quality or speed in the narrow-FoV setting—positioning it as a unified, exact, and efficient renderer for generic cameras. We are equally explicit about the scope of the claim: exactness pertains to projective geometry, while analytic ordering and self-occlusion remain, as in the broader family of ray-based Gaussian renderers, avenues for future work.

ETHICS STATEMENT

This work builds exclusively on publicly available datasets (ScanNet++, ZipNeRF, Aria, and MipNeRF) and involves no human subjects, private data, or personally identifiable information. The methods advance research on 3D scene reconstruction and rendering, with prospective applications in XR, robotics, and autonomous driving. We foresee no direct harmful consequences; we note, however, that the misuse of high-fidelity 3D reconstruction could raise privacy concerns, and to mitigate this we confine our experiments to research datasets already in widespread use within the vision community.

REPRODUCIBILITY STATEMENT

We have taken deliberate steps to make this work reproducible. Every mathematical derivation and proof is provided in full in the Appendix (Secs. B, C, and D)—including the closed-form transmittance integral, the complete backward-gradient chain, and the PBF solver—so that the method can be reimplemented directly from the text. All datasets used in our experiments are publicly available (ScanNet++, ZipNeRF, Aria, and MipNeRF), and the preprocessing pipeline, training protocols, hyperparameters, and evaluation metrics are documented in Sec. G with baseline configurations aligned to their original settings for fair comparison. To further support replication, we will release our full implementation—including the CUDA rasterizer kernels together with training and evaluation scripts—upon publication.

REFERENCES

- Jonathan T. Barron, Ben Mildenhall, Matthew Tancik, Peter Hedman, Ricardo Martin-Brualla, and Pratul P. Srinivasan. Mip-nerf: A multiscale representation for anti-aliasing neural radiance fields. In *IEEE International Conference on Computer Vision (ICCV)*, 2021.
- Jonathan T. Barron, Ben Mildenhall, Dor Verbin, Pratul P. Srinivasan, and Peter Hedman. Zip-nerf: Anti-aliased grid-based neural radiance fields. In *IEEE International Conference on Computer Vision, (ICCV)*, 2023.
- Adam Celarek, George Kopanas, George Drettakis, Michael Wimmer, and Bernhard Kerbl. Does 3d gaussian splatting need accurate volumetric rendering? In *Computer Graphics Forum (CGF)*, 2025.
- David Charatan, Sizhe Lester Li, Andrea Tagliasacchi, and Vincent Sitzmann. pixelsplat: 3d gaussian splats from image pairs for scalable generalizable 3d reconstruction. In *Proceedings of the IEEE/CVF conference on computer vision and pattern recognition (CVPR)*, pp. 19457–19467, 2024.
- Jorge Condor, Sébastien Speierer, Lukas Bode, Aljaz Bozic, Simon Green, Piotr Didyk, and Adrián Jarabo. Don’t splat your gaussians: Volumetric ray-traced primitives for modeling and rendering scattering and emissive media. *ACM Transactions on Graphics (TOG)*, 2025.
- Jonathan Courbon, Youcef Mezouar, Laurent Eckt, and Philippe Martinet. A generic fisheye camera model for robotic applications. In *IEEE/RSJ International Conference on Intelligent Robots and Systems, (IROS)*, pp. 1683–1688, 2007.
- Daniel Duckworth, Peter Hedman, Christian Reiser, Peter Zhizhin, Jean-François Thibert, Mario Lucic, Richard Szeliski, and Jonathan T. Barron. SMERF: streamable memory efficient radiance fields for real-time large-scene exploration. *ACM Transactions on Graphics (TOG)*, 2024.
- Zhongpai Gao, Benjamin Planche, Meng Zheng, Anwesa Choudhuri, Terrence Chen, and Ziyang Wu. 6dgs: Enhanced direction-aware gaussian splatting for volumetric rendering. In *The Thirteenth International Conference on Learning Representations (ICLR)*, 2025.
- Chun Gu, Xiaofei Wei, Zixuan Zeng, Yuxuan Yao, and Li Zhang. Irgs: Inter-reflective gaussian splatting with 2d gaussian ray tracing. In *Proceedings of the Computer Vision and Pattern Recognition Conference (CVPR)*, pp. 10943–10952, 2025.

- Yuliang Guo, Sparsh Garg, S. Mahdi H. Miangoleh, Xinyu Huang, and Liu Ren. Depth any camera: Zero-shot metric depth estimation from any camera. *Conference on Computer Vision and Pattern Recognition (CVPR)*, 2025.
- Florian Hahlbohm, Fabian Friederichs, Tim Weyrich, Linus Franke, Moritz Kappel, Susana Castillo, Marc Stamminger, Martin Eisemann, and Marcus Magnor. Efficient perspective-correct 3d gaussian splatting using hybrid transparency. In *Computer Graphics Forum (CGF)*, 2025.
- Alex Hanson, Allen Tu, Geng Lin, Vasu Singla, Matthias Zwicker, and Tom Goldstein. Speedy-splat: Fast 3d gaussian splatting with sparse pixels and sparse primitives. In *Proceedings of the Computer Vision and Pattern Recognition Conference (CVPR)*, pp. 21537–21546, 2025.
- Qiqi Hou, Randall Rauwendaal, Zifeng Li, Hoang Le, Farzad Farhadzadeh, Fatih Porikli, Alexei Bourd, and Amir Said. Sort-free gaussian splatting via weighted sum rendering. In *The Thirteenth International Conference on Learning Representations (ICLR)*, 2025.
- Binbin Huang, Zehao Yu, Anpei Chen, Andreas Geiger, and Shenghua Gao. 2d gaussian splatting for geometrically accurate radiance fields. In Andres Burbano, Denis Zorin, and Wojciech Jarosz (eds.), *ACM SIGGRAPH Conference Papers*, 2024a.
- Letian Huang, Jiayang Bai, Jie Guo, and Yanwen Guo. Gs++: Error analyzing and optimal gaussian splatting. *CoRR*, 2024b.
- Juho Kannala and Sami S Brandt. A generic camera model and calibration method for conventional, wide-angle, and fish-eye lenses. *IEEE Transactions on Pattern Analysis and Machine Intelligence (TPAMI)*, 2006.
- Bernhard Kerbl, Georgios Kopanas, Thomas Leimkühler, and George Drettakis. 3d gaussian splatting for real-time radiance field rendering. *ACM Transactions on Graphics (TOG)*, 2023.
- Bernhard Kerbl, Andreas Meuleman, Georgios Kopanas, Michael Wimmer, Alexandre Lanvin, and George Drettakis. A hierarchical 3d gaussian representation for real-time rendering of very large datasets. *ACM Transactions on Graphics (TOG)*, 43(4):1–15, 2024.
- Leonid Keselman and Martial Hebert. Approximate differentiable rendering with algebraic surfaces. In *European Conference on Computer Vision (ECCV)*, 2022.
- Shakiba Kheradmand, Daniel Rebain, Gopal Sharma, Weiwei Sun, Yang-Che Tseng, Hossam Isack, Abhishek Kar, Andrea Tagliasacchi, and Kwang Moo Yi. 3d gaussian splatting as markov chain monte carlo. *Advances in Neural Information Processing Systems (NeurIPS)*, 37:80965–80986, 2024.
- Varun Ravi Kumar, Ciarán Eising, Christian Witt, and Senthil Kumar Yogamani. Surround-view fisheye camera perception for automated driving: Overview, survey & challenges. *IEEE Transactions on Intelligent Transportation Systems*, 24(4):3638–3659, 2023.
- Zimu Liao, Siyan Chen, Rong Fu, Yi Wang, Zhongling Su, Hao Luo, Li Ma, Linning Xu, Bo Dai, Hengjie Li, et al. Fisheye-gs: Lightweight and extensible gaussian splatting module for fisheye cameras. *ECCV Workshop*, 2024.
- Changkun Liu, Shuai Chen, Yash Sanjay Bhalgat, Siyan HU, Ming Cheng, Zirui Wang, Victor Adrian Prisacariu, and Tristan Braud. Gs-cpr: Efficient camera pose refinement via 3d gaussian splatting. In *The Thirteenth International Conference on Learning Representations (ICLR)*, 2025.
- Tao Lu, Mulin Yu, Linning Xu, Yuanbo Xiangli, Limin Wang, Dahua Lin, and Bo Dai. Scaffold-gs: Structured 3d gaussians for view-adaptive rendering. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, pp. 20654–20664, 2024.
- Zhaoyang Lv, Maurizio Monge, Ka Chen, Yufeng Zhu, Michael Goesele, Jakob Engel, Zhao Dong, and Richard Newcombe. Photoreal scene reconstruction from an egocentric device. In *ACM SIGGRAPH Conference Papers*, 2025.

- Alexander Mai, Peter Hedman, George Kopanas, Dor Verbin, David Futschik, Qiangeng Xu, Falko Kuester, Jonathan T Barron, and Yinda Zhang. EVER: exact volumetric ellipsoid rendering for real-time view synthesis. *IEEE International Conference on Computer Vision (ICCV)*, 2025.
- Saswat Subhajyoti Mallick, Rahul Goel, Bernhard Kerbl, Markus Steinberger, Francisco Vicente Carrasco, and Fernando De La Torre. Taming 3dgs: High-quality radiance fields with limited resources. In *SIGGRAPH Asia Conference Papers*, 2024.
- Xiaoyang Mao, Lichan Hong, and Arie Kaufman. Splatting of curvilinear volumes. In *Proceedings Visualization'95*, pp. 61–68. IEEE, 1995.
- Christopher Mei and Patrick Rives. Single view point omnidirectional camera calibration from planar grids. In *IEEE International Conference on Robotics and Automation (ICRA)*, 2007.
- Nicolas Moënné-Loccoz, Ashkan Mirzaei, Or Perel, Riccardo de Lutio, Janick Martinez Esturo, Gavriel State, Sanja Fidler, Nicholas Sharp, and Zan Gojcic. 3d gaussian ray tracing: Fast tracing of particle scenes. *ACM Transactions on Graphics (TOG)*, 2024.
- Hazem Rashed, Eslam Mohamed, Ganesh Sistu, Varun Ravi Kumar, Ciaran Eising, Ahmad El-Sallab, and Senthil Yogamani. Generalized object detection on fisheye cameras for autonomous driving: Dataset, representations and baseline. In *Proceedings of the IEEE/CVF Winter Conference on Applications of Computer Vision, (WACV)*, pp. 2272–2280, 2021.
- Liu Ren, Hanspeter Pfister, and Matthias Zwicker. Object space ewa surface splatting: A hardware accelerated approach to high quality point rendering. In *Computer graphics forum (CGF)*, 2002.
- Samuel Rota Bulò, Lorenzo Porzi, and Peter Kotschieder. Revising densification in gaussian splatting. In *European Conference on Computer Vision (ECCV)*, pp. 347–362. Springer, 2024.
- Johannes Lutz Schönberger and Jan-Michael Frahm. Structure-from-motion revisited. In *Conference on Computer Vision and Pattern Recognition (CVPR)*, 2016.
- Ahmed Rida Sekkat, Yohan Dupuis, Varun Ravi Kumar, Hazem Rashed, Senthil Yogamani, Pascal Vasseur, and Paul Honeine. Synwoodscape: Synthetic surround-view fisheye camera dataset for autonomous driving. *IEEE Robotics and Automation Letters (RAL)*, 7(3):8502–8509, 2022.
- Chinmay Talegaonkar, Yash Belhe, Ravi Ramamoorthi, and Nicholas Antipa. Volumetrically consistent 3d gaussian rasterization. In *Proceedings of the Computer Vision and Pattern Recognition Conference (CVPR)*, pp. 10953–10963, 2025.
- Lee Westover. Footprint evaluation for volume rendering. In *Proceedings of the 17th annual conference on Computer graphics and interactive techniques*, pp. 367–376, 1990.
- Qi Wu, Janick Martinez Esturo, Ashkan Mirzaei, Nicolas Moënné-Loccoz, and Zan Gojcic. 3dgt: Enabling distorted cameras and secondary rays in gaussian splatting. *Conference on Computer Vision and Pattern Recognition (CVPR)*, 2025.
- Zhiwen Yan, Weng Fei Low, Yu Chen, and Gim Hee Lee. Multi-scale 3d gaussian splatting for anti-aliased rendering. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition, (CVPR)*, pp. 20923–20931, 2024.
- Chandan Yeshwanth, Yueh-Cheng Liu, Matthias Nießner, and Angela Dai. Scannet++: A high-fidelity dataset of 3d indoor scenes. In *Proceedings of the IEEE/CVF International Conference on Computer Vision (ICCV)*, 2023.
- Senthil Yogamani, Ciarán Hughes, Jonathan Horgan, Ganesh Sistu, Padraig Varley, Derek O’Dea, Michal Uricár, Stefan Milz, Martin Simon, Karl Amende, et al. Woodscape: A multi-task, multi-camera fisheye dataset for autonomous driving. In *Proceedings of the IEEE/CVF International Conference on Computer Vision (ICCV)*, pp. 9308–9318, 2019.
- Senthil Yogamani, David Unger, Venkatraman Narayanan, and Varun Ravi Kumar. Fisheyebevseg: Surround view fisheye cameras based bird’s-eye view segmentation for autonomous driving. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition, (CVPR)*, pp. 1331–1334, 2024.

-
- Zehao Yu, Anpei Chen, Binbin Huang, Torsten Sattler, and Andreas Geiger. Mip-splatting: Alias-free 3d gaussian splatting. In *Proceedings of the IEEE/CVF conference on computer vision and pattern recognition, (CVPR)*, pp. 19447–19456, 2024a.
- Zehao Yu, Torsten Sattler, and Andreas Geiger. Gaussian opacity fields: Efficient adaptive surface reconstruction in unbounded scenes. *ACM Transactions on Graphics (TOG)*, 2024b.
- Zheng Zhang, Wenbo Hu, Yixing Lao, Tong He, and Hengshuang Zhao. Pixel-gs: Density control with pixel-aware gradient for 3d gaussian splatting. In *European Conference on Computer Vision (ECCV)*, pp. 326–342. Springer, 2024.
- Zichao Zhang, Henri Rebecq, Christian Forster, and Davide Scaramuzza. Benefit of large field-of-view cameras for visual odometry. In *2016 IEEE International Conference on Robotics and Automation (ICRA)*, pp. 801–808. IEEE, 2016.
- Hexu Zhao, Haoyang Weng, Daohan Lu, Ang Li, Jinyang Li, Aurojit Panda, and Saining Xie. On scaling up 3d gaussian splatting training. In *The Thirteenth International Conference on Learning Representations (ICLR)*, 2025.
- Matthias Zwicker, Hanspeter Pfister, Jeroen Van Baar, and Markus Gross. Ewa volume splatting. In *Proceedings Visualization, 2001. VIS'01.*, pp. 29–538. IEEE, 2001.

CONTENTS

1 Introduction	1
2 Related Works	2
2.1 Splatting-Based Gaussian Rendering	2
2.2 Ray-Based Gaussian Rendering	3
3 Method	4
3.1 Rendering with Exact Projective Geometry	5
3.2 Ray-Particle Association with Exact Projective Geometry	5
3.3 Bipolar Equiangular Projection (BEAP)	6
4 Experiments	7
4.1 Scaling Gaussians vs. Projective Exactness in Wide-FoV Cameras	8
4.2 Comprehensive Evaluation on All-FoV Novel View Synthesis	8
4.3 Comprehensive Runtime Analysis	10
4.4 Local Affine Approximation vs. Projective-Exact Integral	10
4.5 BEAP On/Off Study	11
5 Conclusions	11
a Ethics Statement	13
b Reproducibility Statement	13
c LLM Usage Statement	19
A Appendix: Exactness and Limitation	19
B Appendix: Ray-Gaussian Transmittance with Projective Exactness	19
B.1 Preliminaries And Notations	19
B.2 Differentiable Rendering with Projective Exactness	20
B.3 Ray Integral of Isotropic 3D Gaussian	21
C Appendix: Cookbook of Gradient Computation	22
C.1 Chain Rule	22
C.2 Two-Stage Propagation	22
C.3 Gradients for Canonical Ray	22
C.4 Gradients for Whitening Matrix	23
C.5 Gradients for Scaling and Rotation	23
C.6 Gradients for 3D Mean	23
C.7 Numerical Stability of Our Differentiable Framework	24

918	D Appendix: Additional Details in Association	24
919	D.1 Particle Bounding Frustum (PBF) Solver	25
920	D.2 Details in Computing CSF-PBF Intersection	25
921	D.3 Relation to Prior Works	26
922		
923	E Appendix: BEAP Image Inverse Interpolation	27
924	Pinhole Camera Model	27
925	KB Fisheye Model	28
926		
927	F Appendix: High-level Algorithmic Summary	28
928		
929	G Appendix: Additional Implementation Details	28
930	Baselines	28
931	Implementation Details	28
932	Central and peripheral regions in ScanNet++ evaluation	29
933	Cross-camera generalization evaluation on ZipNeRF	29
934		
935	H Appendix: EWA Splatting Preliminaries	30
936	From camera coordinates to ray coordinates	30
937	Local affine approximation	30
938	Splatting approximation error	30
939		
940	I Appendix: Advantage over Prior Ray-Tracing Methods	31
941		
942	J Appendix: Results on Egocentric Dataset	31
943		
944	K Appendix: Additional Material and Results	33
945	K.1 Additional Material.	33
946	K.2 Additional Results.	33
947		
948		
949		
950		
951		
952		
953		
954		
955		
956		
957		
958		
959		
960		
961		
962		
963		
964		
965		
966		
967		
968		
969		
970		
971		

LLM USAGE STATEMENT

We used a large language model (ChatGPT, OpenAI) as a writing assistant. Specifically, the LLM was employed to (i) improve the clarity and readability of the manuscript, (ii) suggest stylistic edits to figure captions and section transitions, and (iii) provide alternative phrasings for technical descriptions without altering the scientific content.

The LLM was not involved in research ideation, algorithm design, data analysis, or experimental validation. All technical contributions, mathematical derivations, and results are the work of the authors. The authors take full responsibility for the content of this paper.

A APPENDIX: EXACTNESS AND LIMITATION

We state precisely, and deliberately narrowly, what our exactness claim entails. It refers to the *elimination of the linear approximation of projective geometry* in both the “rendering” (transmittance) and the “ray-particle association” stages—and to that alone. The defining consequence is that, unlike splatting-based methods, the residual projective error does not shrink as the Gaussian count grows; it is removed outright (see Sec. 4.1). We explicitly do *not* claim exactness in the full physical volumetric sense: the framework still approximates the ordering of overlapping Gaussians, their mutual self-attenuation, and their overlap, and we regard these—rather than any projective residual—as its principal limitations. This distinction is what the main text’s use of “exact” should be read against: exact in projective geometry, approximate in volumetric compositing.

Limitations. Because our formulation integrates each Gaussian independently, it does not explicitly resolve self-occlusion, and it handles ordering through depth-based sorting; as the supplementary videos show, this can occasionally surface as popping artifacts. To our knowledge, EVER (Mai et al., 2025) is the sole method that resolves ordering and self-occlusion analytically, and it does so only by approximating each Gaussian as a constant-density ellipsoid—at a substantial cost in rendering speed and representational capacity.

Crucially, this limitation is not peculiar to our method. Prior ray-tracing-based Gaussian renderers—Fuzzy Metaballs (Keselman & Hebert, 2022) and its successors, including 3DGR/UT (Moëne-Loccoz et al., 2024; Wu et al., 2025)—likewise rest on the “maximum-response” assumption for ray-Gaussian interaction, which by construction disregards ordering and self-attenuation, yet this assumption has been shown to introduce negligible error in practice (see Celarek et al. (2025), Fig. 9–11 / Sec. 7.1), independently of the number of Gaussians. Our depth-based ordering therefore attains the same practical accuracy as existing ray-tracing-based Gaussian renderers; the limitation is a property of the family as a whole, not a shortcoming introduced by 3DGEER.

Future work. Although the practical impact of ordering and self-occlusion is small in current Gaussian-based ray tracing, both factors nonetheless bound exact volumetric correctness, and we regard closing this gap as the natural next step. One promising direction is to incorporate analytic or learnable self-attenuation models—drawing on ellipsoidal integration (e.g., EVER (Mai et al., 2025)) while avoiding its heavy runtime cost. A second is to treat ordering more principledly, which could further suppress the rare popping artifacts observed under extreme configurations. Both directions are orthogonal to our contribution on projective exactness, and resolving them would extend the framework’s robustness to large-scale and high-dynamic scenes—settings we have not yet evaluated and which lie beyond the static novel-view-synthesis scope of the present study.

B APPENDIX: RAY-GAUSSIAN TRANSMITTANCE WITH PROJECTIVE EXACTNESS

B.1 PRELIMINARIES AND NOTATIONS

We adopt the classical parameterization of 3D Gaussian primitives established in prior work (Zwicker et al., 2001). Each 3D Gaussian is specified by a mean position $\mu \in \mathbb{R}^3$ and a

3D covariance matrix Σ in the world coordinate system, giving the density:

$$\mathcal{G}_{\Sigma, \mu}(\mathbf{x}) = \frac{1}{\rho |\Sigma|^{1/2}} \exp \left(-\frac{1}{2} (\mathbf{x} - \mu)^\top \Sigma^{-1} (\mathbf{x} - \mu) \right), \quad (\text{B.1})$$

where the normalization constant is $\rho = \sqrt{2\pi}$ and $\mathbf{x} \in \mathbb{R}^3$ denotes a world-space point. This formulation departs deliberately from recent methods such as 3DGS (Kerbl et al. 2023) and 3DGUT (Wu et al. 2025), which drop the determinant term $|\Sigma|^{1/2}$ to simplify splatting or association; retaining it is precisely what yields our clean closed-form integral, since the determinant cancels against the Jacobian of the canonical change of variables. The covariance matrix factorizes as $\Sigma = R S S^\top R^\top$, with scaling matrix $S = \text{diag}(\mathbf{s})$ built from the scaling vector $\mathbf{s} \in \mathbb{R}^3$, and rotation matrix $R \in \text{SO}(3)$ obtained from a unit quaternion $\mathbf{q} \in \mathbb{R}^4$.

The transmittance contributed by a single 3D Gaussian particle along a ray $\mathbf{r}(t) = \mathbf{o} + t\mathbf{d}$ is given by:

$$T(\mathbf{o}, \mathbf{d}) = \sigma \int_{t \in \mathbb{R}} \mathcal{G}_{\Sigma, \mu}(\mathbf{o} + t\mathbf{d}) dt, \quad (\text{B.2})$$

where σ is the opacity coefficient of the Gaussian.

Finally, for a depth-sorted set of 3D Gaussian particles contributing to a given ray, the rendered color C follows from front-to-back α -compositing:

$$C(\mathbf{o}, \mathbf{d}) = \sum_i c_i(\mathbf{o}, \mu_i) T_i \prod_{j=1}^{i-1} (1 - T_j), \quad (\text{B.3})$$

where $c_i(\mathbf{o}, \mu_i)$ is the view-dependent color of the i -th Gaussian as seen from the optical center $\mathbf{o}$, parameterized by spherical harmonics (SH) (Kerbl et al. 2023). Minimizing the reconstruction loss between the rendered and observed colors along each ray jointly optimizes, through backpropagation, the geometric parameters $\mathbf{s}, \mathbf{q}, \mu$, the opacity coefficient σ , and the SH coefficients.

B.2 DIFFERENTIABLE RENDERING WITH PROJECTIVE EXACTNESS

The core challenge in projective-exact Gaussian rendering within a ray-marching formulation is to obtain a closed-form solution for integrating Gaussian density and color along a 3D ray. We meet it in two steps: relative to each Gaussian, we first transform the ray into a canonical coordinate system in which the transmittance integral (Eq. B.2) is preserved and every Gaussian becomes isotropic, and we then derive the exact ray color from first principles within that frame.

Concretely, we fix a single canonical coordinate system shared across all 3D Gaussian primitives. For each Gaussian, a point $\mathbf{u} \in \mathbb{R}^3$ in canonical space maps to world coordinates via:

$$\begin{bmatrix} \mathbf{x} \\ 1 \end{bmatrix} = H \begin{bmatrix} \mathbf{u} \\ 1 \end{bmatrix} = \begin{bmatrix} RS & \mu \\ \mathbf{0} & 1 \end{bmatrix} \begin{bmatrix} \mathbf{u} \\ 1 \end{bmatrix}, \quad (\text{B.4})$$

Under this transformation, an infinitesimal segment $\Delta \mathbf{u}$ along a ray in the canonical frame maps to a segment $\Delta \mathbf{x} = RS \Delta \mathbf{u}$ in the world frame.

To guarantee that the transmittance contributed by each Gaussian is preserved across the two coordinate systems, we require the elementary transmittance—the product of density and segment length—to remain invariant under the transformation (see Fig. 2 Right):

$$\mathcal{G}_{\mathbf{I}, \mathbf{o}}(\mathbf{u}) |\Delta \mathbf{u}| = \mathcal{G}_{\Sigma, \mu}(\mathbf{x}) |\Delta \mathbf{x}|. \quad (\text{B.5})$$

This constraint absorbs the Jacobian determinant into the measure, since $|\frac{\Delta \mathbf{x}}{\Delta \mathbf{u}}| = |RS| = |\Sigma|^{1/2}$. The canonical density that results is therefore:

$$\mathcal{G}_{\mathbf{I}, \mathbf{o}}(\mathbf{u}) = \frac{1}{\rho} \exp \left(-\frac{1}{2} \mathbf{u}^\top \mathbf{u} \right), \quad (\text{B.6})$$

which is isotropic and shared by every primitive in canonical space.

The consequence is decisive: the exact closed-form transmittance integral of each Gaussian along the ray $\mathbf{r}(t)$ reduces to the accumulated density along the transformed ray $\mathbf{r}_u(t) = \mathbf{o}_u + t\mathbf{d}_u$ within

the canonical isotropic Gaussian. The contribution from $t < 0$ is negligible unless the optical center lies inside the 3-sigma contour—a configuration that every 3D Gaussian rendering framework culls during optimization.

This integral admits a *closed-form* solution in terms of the perpendicular distance from the ray to the Gaussian center, as stated in Theorem 1.

$$T(\mathbf{o}, \mathbf{d}) = \sigma \int_{t \in \mathbb{R}} \mathcal{G}_{\mathbf{I}, \mathbf{o}}(\mathbf{o}_u + t \mathbf{d}_u) dt = \sigma \exp\left(-\frac{1}{2} (D_{\mu, \Sigma}(\mathbf{o}, \mathbf{d}))^2\right), \quad (\text{B.7})$$

where $D_{\mu, \Sigma}(\mathbf{o}, \mathbf{d})$ is the perpendicular distance from the Gaussian center (the canonical-space origin) to the transformed ray $\mathbf{r}_u(t)$. Geometrically, this quantity is exactly the minimal *Mahalanobis distance* from any point on the ray to the Gaussian in world space—a direct measure of how closely the ray passes the particle distribution.

It remains only to compute the perpendicular distance from the canonical origin to the transformed ray. We write the transformed ray’s direction vector $\mathbf{d}_u$ and moment vector $\mathbf{m}_u$ as:

$$\mathbf{d}_u = (S^{-1} R^\top) \mathbf{d}, \quad \mathbf{m}_u = \mathbf{o}_u \times \mathbf{d}_u, \quad (\text{B.8})$$

where $\mathbf{o}_u = (S^{-1} R^\top) (\mathbf{o} - \mu)$ is the optical center in canonical space. From these, the square of $D_{\mu, \Sigma}(\mathbf{o}, \mathbf{d})$ follows efficiently as:

$$(D_{\mu, \Sigma}(\mathbf{o}, \mathbf{d}))^2 = \frac{\mathbf{m}_u^\top \mathbf{m}_u}{\mathbf{d}_u^\top \mathbf{d}_u}. \quad (\text{B.9})$$

Substituting this squared distance into Eq. B.7 and then the resulting transmittance into Eq. B.3 yields the accumulated color along the ray—an end-to-end closed form free of any projective approximation.

B.3 RAY INTEGRAL OF ISOTROPIC 3D GAUSSIAN

Theorem 1. *For a ray and a standard isotropic 3D Gaussian, the ray–Gaussian integral admits a closed form proportional to*

$$\exp\left(-\frac{1}{2} D^2\right),$$

where D is the perpendicular distance from the Gaussian center to the ray.

Proof. We first translate the coordinate system so that the Gaussian $\mathcal{G}$ is centered at the origin, translating the ray $\mathbf{r}_u$ accordingly, and then rotate the scene to align the ray direction with the z -axis. By the translation-invariance of the integral together with the isotropy of the standard Gaussian, neither transformation alters the value of the integral.

The translated standard isotropic 3D Gaussian is defined as:

$$\mathcal{G}_{\mathbf{I}, \mathbf{o}}(\mathbf{u}) = \frac{1}{\rho} \exp\left(-\frac{1}{2} \mathbf{u}^\top \mathbf{u}\right), \quad (\text{B.10})$$

which matches Eq. B.6. Let the z -axis aligned ray as:

$$\mathbf{r}_u(t) = \begin{bmatrix} \mathbf{o}_u \\ \mathbf{o}_v \\ 0 \end{bmatrix} + t \begin{bmatrix} 0 \\ 0 \\ 1 \end{bmatrix}, \quad (\text{B.11})$$

the ray–Gaussian integral becomes:

$$\begin{aligned} \int_{t \in \mathbb{R}} \mathcal{G}_{\mathbf{I}, \mathbf{o}}(\mathbf{r}_u(t)) dt &= \frac{1}{\rho} \exp\left(-\frac{u_0^2 + v_0^2}{2}\right) \int_{t \in \mathbb{R}} \exp\left(-\frac{1}{2} t^2\right) dt \\ &= \frac{\sqrt{2\pi}}{\rho} \exp\left(-\frac{1}{2} D^2\right), \end{aligned} \quad (\text{B.12})$$

where $D^2 = \frac{u_0^2 + v_0^2}{2}$ denotes the squared perpendicular distance from the ray to the Gaussian center in canonical space. Hence the integral is proportional to $\exp\left(-\frac{1}{2} D^2\right)$, as claimed. $\square$

The density shared by our 3D primitives in canonical space (Eq. B.6) has exactly the isotropic form invoked above. For a world-coordinate ray $(\mathbf{o}, \mathbf{d})$, the argument D depends in addition on the world-space Gaussian parameters μ, Σ , which determine the transformation of the ray into canonical space. The transmittance of each Gaussian contributing to the ray therefore reads:

$$T = \frac{\sqrt{2\pi}\sigma}{\rho} \exp\left(-\frac{1}{2}(D_{\mu,\Sigma}(\mathbf{o}, \mathbf{d}))^2\right), \quad (\text{B.13})$$

which recovers Eq. B.7 for the choice $\rho = \sqrt{2\pi}$.

C APPENDIX: COOKBOOK OF GRADIENT COMPUTATION

C.1 CHAIN RULE

Recall that $T = \sigma\alpha$ is the ray-Gaussian transmittance (Eq. B.7), with σ the opacity coefficient and α the ray-Gaussian integral. Let $\kappa = D_{\mu,\Sigma}^2$ denote the minimal squared Mahalanobis distance from the ray to the Gaussian distribution, S the Gaussian scaling, and R the world-space rotation matrix. Given the world-space covariance $\Sigma = RSS^\top R^\top$, the PCA whitening matrix W_{pca} satisfies:

$$W_{\text{pca}} = \Sigma^{-\frac{1}{2}} = S^{-1}R^\top, \quad (\text{C.1})$$

where $S^{-1} = \text{diag}\left(\frac{1}{s}\right)$.

As Fig. 2 illustrates, the chain rule yields the derivatives *w.r.t.* the Gaussian parameters—the scaling vector $\mathbf{s}$, the rotation quaternion $\mathbf{q}$, and the mean μ :

$$\frac{\partial T}{\partial \mathbf{s}} = \frac{\partial T}{\partial \kappa} \frac{\partial \kappa}{\partial W_{\text{pca}}} \frac{\partial W_{\text{pca}}}{\partial \mathbf{s}}, \quad (\text{C.2})$$

$$\frac{\partial T}{\partial \mathbf{q}} = \frac{\partial T}{\partial \kappa} \frac{\partial \kappa}{\partial W_{\text{pca}}} \frac{\partial W_{\text{pca}}}{\partial \mathbf{q}}, \quad (\text{C.3})$$

$$\frac{\partial T}{\partial \mu} = \frac{\partial T}{\partial \kappa} \frac{\partial \kappa}{\partial \mu}. \quad (\text{C.4})$$

C.2 TWO-STAGE PROPAGATION

To minimize overhead in the shared propagation, we structure the gradient computation as the two-stage pipeline of 3DGS (Kerbl et al. 2023), comprising RenderCUDA (RC) and PreprocessCUDA (PC). The former performs ray-wise accumulation, accelerated by our Particle Bounding Frustum (PBF) association (Sec. 3.2); the latter operates at Gaussian-wise complexity and exploits view-frustum culling for an additional speedup. Partitioning the gradient in this way keeps the expensive per-ray work minimal while relegating per-Gaussian terms to the cheaper stage.

From Eq. B.7, the gradient *w.r.t.* the opacity coefficient and the shared partial derivative for the Gaussian parameters in RC are, respectively:

$$\left.\frac{\partial T}{\partial \sigma}\right|_{\text{RC}} = \alpha, \quad \left.\frac{\partial T}{\partial \kappa}\right|_{\text{RC}} = -\frac{T}{2}. \quad (\text{C.5})$$

The remaining gradients to the Gaussian parameters then back-propagate from the squared distance, namely $\frac{\partial \kappa}{\partial \mathbf{s}}, \frac{\partial \kappa}{\partial \mathbf{q}}, \frac{\partial \kappa}{\partial \mu}$.

C.3 GRADIENTS FOR CANONICAL RAY

We next backpropagate gradients to the canonical ray parameters—the origin $\mathbf{o}_u$, direction $\mathbf{d}_u$, and moment $\mathbf{m}_u = \mathbf{o}_u \times \mathbf{d}_u$ of Eq. B.8. These three quantities jointly determine the Mahalanobis distance in Eq. B.9 and hence encode the complete canonical ray geometry.

$$\left. \frac{\partial \kappa}{\partial \mathbf{m}_u} \right|_{\text{RC}} = \left(\frac{2}{\mathbf{d}_u^\top \mathbf{d}_u} \right) \mathbf{m}_u, \quad (\text{C.6})$$

$$\left. \frac{\partial \kappa}{\partial \mathbf{o}_u} \right|_{\text{RC}} = \frac{\partial \kappa}{\partial \mathbf{m}_u} \times \mathbf{d}_u, \quad (\text{C.7})$$

$$\left. \frac{\partial \kappa}{\partial \mathbf{d}_u} \right|_{\text{RC}} = - \left(\frac{2\kappa}{\mathbf{d}_u^\top \mathbf{d}_u} \right) \mathbf{d}_u + \mathbf{o}_u \times \frac{\partial \kappa}{\partial \mathbf{m}_u}. \quad (\text{C.8})$$

C.4 GRADIENTS FOR WHITENING MATRIX

Recall that $\mathbf{d}_u = \mathbf{W}_{\text{pca}} \mathbf{d}$ and $\mathbf{o}_u = \mathbf{W}_{\text{pca}} (\mathbf{o} - \boldsymbol{\mu})$, where $\mathbf{o} = -R_c^\top \mathbf{t}_c$ is the world-space optical center and $\mathbf{d}$ the ray direction. With the gradients *w.r.t.* the ray parameters now in hand, we propagate them to the PCA whitening matrix $\mathbf{W}_{\text{pca}}$ through outer products with the corresponding world-space vectors:

$$\begin{aligned} \text{RenderCUDA: } \left. \frac{\partial \kappa}{\partial \mathbf{W}_{\text{pca}}} \right|_{\text{RC}} &= \frac{\partial \kappa}{\partial \mathbf{d}_u} \mathbf{d}^\top; \\ \text{PreprocessCUDA: } \left. \frac{\partial \kappa}{\partial \mathbf{W}_{\text{pca}}} \right|_{\text{PC}} &= \frac{\partial \kappa}{\partial \mathbf{o}_u} (\mathbf{o} - \boldsymbol{\mu})^\top; \\ \text{Overall: } \left. \frac{\partial \kappa}{\partial \mathbf{W}_{\text{pca}}} \right| &= \left. \frac{\partial \kappa}{\partial \mathbf{W}_{\text{pca}}} \right|_{\text{RC}} + \left. \frac{\partial \kappa}{\partial \mathbf{W}_{\text{pca}}} \right|_{\text{PC}}. \end{aligned} \quad (\text{C.9})$$

Here the gradient through $\mathbf{d}_u$ is accumulated ray-wise in `RenderCUDA`, whereas the gradient through $\mathbf{o}_u$ is computed Gaussian-wise in `PreprocessCUDA`; the sum of the two components constitutes the complete gradient *w.r.t.* the PCA transformation matrix $\mathbf{W}_{\text{pca}}$.

C.5 GRADIENTS FOR SCALING AND ROTATION

Thus, for scaling gradients $\frac{\partial \kappa}{\partial \mathbf{s}} = \frac{\partial \kappa}{\partial \mathbf{W}_{\text{pca}}} \frac{\partial \mathbf{W}_{\text{pca}}}{\partial \mathbf{s}}$, we have:

$$\left. \frac{\partial \mathbf{W}_{\text{pca}}^{i,j}}{\partial s_k} \right|_{\text{PC}} = \begin{cases} -\frac{1}{s_k^2} R_{j,k} & \text{if } i = k \\ 0 & \text{otherwise} \end{cases}. \quad (\text{C.10})$$

To derive gradients for rotation $\frac{\partial \kappa}{\partial \mathbf{q}} = \frac{\partial \kappa}{\partial \mathbf{W}_{\text{pca}}} \frac{\partial \mathbf{W}_{\text{pca}}}{\partial \mathbf{q}}$, given the quaternion form $\mathbf{q} = q_r + q_i \mathbf{i} + q_j \mathbf{j} + q_k \mathbf{k}$, we can write the whitening matrix as:

$$\mathbf{W}_{\text{pca}} = 2 \begin{pmatrix} \frac{1}{s_1} (\frac{1}{2} - q_j^2 - q_k^2) & \frac{1}{s_1} (q_i q_j + q_r q_k) & \frac{1}{s_1} (q_i q_k - q_r q_j) \\ \frac{1}{s_2} (q_i q_j - q_r q_k) & \frac{1}{s_2} (\frac{1}{2} - q_i^2 - q_k^2) & \frac{1}{s_2} (q_j q_k + q_r q_i) \\ \frac{1}{s_3} (q_i q_k + q_r q_j) & \frac{1}{s_3} (q_j q_k - q_r q_i) & \frac{1}{s_3} (\frac{1}{2} - q_i^2 - q_j^2) \end{pmatrix}. \quad (\text{C.11})$$

As a result, we derive the following end gradients for the quaternion $\mathbf{q}$ as:

$$\begin{aligned} \left. \frac{\partial \mathbf{W}_{\text{pca}}}{\partial q_r} \right|_{\text{PC}} &= 2 \begin{pmatrix} 0 & \frac{q_k}{s_1} & -\frac{q_j}{s_1} \\ -\frac{q_k}{s_2} & 0 & \frac{q_i}{s_2} \\ \frac{q_j}{s_3} & -\frac{q_i}{s_3} & 0 \end{pmatrix}, \quad \left. \frac{\partial \mathbf{W}_{\text{pca}}}{\partial q_i} \right|_{\text{PC}} = 2 \begin{pmatrix} 0 & \frac{q_j}{s_1} & \frac{q_k}{s_1} \\ \frac{q_j}{s_2} & -\frac{2q_i}{s_2} & \frac{q_r}{s_2} \\ \frac{q_k}{s_3} & -\frac{q_r}{s_3} & -\frac{2q_i}{s_3} \end{pmatrix}, \\ \left. \frac{\partial \mathbf{W}_{\text{pca}}}{\partial q_j} \right|_{\text{PC}} &= 2 \begin{pmatrix} -\frac{2q_j}{s_1} & \frac{q_i}{s_1} & -\frac{q_r}{s_1} \\ \frac{q_i}{s_2} & 0 & \frac{q_k}{s_2} \\ \frac{q_r}{s_3} & \frac{q_k}{s_3} & -\frac{2q_j}{s_3} \end{pmatrix}, \quad \left. \frac{\partial \mathbf{W}_{\text{pca}}}{\partial q_k} \right|_{\text{PC}} = 2 \begin{pmatrix} -\frac{2q_k}{s_1} & \frac{q_r}{s_1} & \frac{q_i}{s_1} \\ -\frac{q_r}{s_2} & -\frac{2q_k}{s_2} & \frac{q_j}{s_2} \\ \frac{q_i}{s_3} & \frac{q_j}{s_3} & 0 \end{pmatrix}. \end{aligned} \quad (\text{C.12})$$

C.6 GRADIENTS FOR 3D MEAN

In parallel, the end gradient *w.r.t.* the Gaussian 3D mean $\boldsymbol{\mu}$ is:

$$\left. \frac{\partial \kappa}{\partial \boldsymbol{\mu}} \right|_{\text{PC}} = -\mathbf{W}_{\text{pca}}^\top \frac{\partial \kappa}{\partial \mathbf{o}_u}. \quad (\text{C.13})$$



Figure C.1: **Artifacts without 3D Filtering in GOF.** (Left) the artifacts that arise when degenerate Gaussians are left unfiltered in GOF; (Right) the stable, artifact-free result obtained under our forward and backward formulations, with no filtering applied.

C.7 NUMERICAL STABILITY OF OUR DIFFERENTIABLE FRAMEWORK

The choice of forward formulation, backward gradient computation, and intermediate cached variables is decisive for numerical stability, and it is here that our derivation pays a further dividend. Prior approaches such as GOF (Yu et al., 2024b) exhibit instability in backward propagation and, in consequence, must resort to auxiliary 3D filtering heuristics to keep training stable. Our closed-form formulation, together with its exact gradient chain, precludes this instability by construction and thus dispenses with any such filtering.

The essential difference lies in how the two families expand $\mathbf{m}_u^\top \mathbf{m}_u$. In exact arithmetic, the following two forms are geometrically equivalent:

$$\mathbf{m}_u^\top \mathbf{m}_u = \|\mathbf{o}_u\|^2 \|\mathbf{d}_u\|^2 - (\mathbf{o}_u^\top \mathbf{d}_u)^2. \quad (\text{C.14})$$

When one axis of a 3D Gaussian is scaled close to zero, however, the canonical-space vectors $\mathbf{o}_u$ and $\mathbf{d}_u$ become nearly parallel: $\cos(\angle(\mathbf{o}_u, \mathbf{d}_u)) \rightarrow 1$ while both $\|\mathbf{o}_u\|$ and $\|\mathbf{d}_u\|$ diverge. The difference $\|\mathbf{o}_u\|^2 \|\mathbf{d}_u\|^2 - (\mathbf{o}_u^\top \mathbf{d}_u)^2$ is then acutely susceptible to catastrophic cancellation and overflow—in practice even returning negative values—which destabilizes backward propagation.

Fig. C.1 (Left) makes the resulting artifacts in GOF explicit: absent additional 3D filtering to remove degenerate Gaussians, severe artifacts appear. The snowflake noise originates in the overflow of Eq. C.14 where premature negative values and vanishingly small positive values together amplify the Gaussian intensity. Because the EWA approximation—with its Jacobian and conic—remains numerically stable, these artifacts trace primarily to inaccurate maximum-response computation, the same mechanism that produces the grid-line patterns.

Our CUDA rasterizer, by contrast, computes the cross product and its gradients directly (Eq. B.9 for the forward pass and Eqs. C.6–C.9 for the backward pass), removing the instability at its source: the forward pass is artifact-free and the backward pass yields accurate gradients. Fig. C.1 (Right) confirms this directly—applying our forward and backward formulations to GOF eliminates the numerical-instability artifacts even without any 3D filtering of degenerate (flat disk-like or needle) Gaussians.

D APPENDIX: ADDITIONAL DETAILS IN ASSOCIATION

Section D.1 presents the detailed derivation; Section D.2 sets out the implementation considerations required for a stable PBF under large-FoV and omnidirectional cameras; and Section D.3 clarifies the derivational relationships to prior work and states precisely the conditions under which earlier formulas coincide with ours.

1296
1297

Recalling Eq. 8 let $P = (VH)^T \in \mathbb{R}^{4 \times 4}$ denote the plane transformation matrix from view space to canonical space. Taking the unknown bounds $c = \tan \theta_{1,2}$ as the representative case:

$$\mathbf{g}_u = -P_0 + cP_2 = \begin{bmatrix} -P_{00} + cP_{02} \\ -P_{10} + cP_{12} \\ -P_{20} + cP_{22} \\ -P_{30} + cP_{32} \end{bmatrix}, \quad (\text{D.1})$$

where P_i denotes the i -th row of the matrix, written as a 4×1 column vector, and P_{ij} its j -th element. Recalling Eq. 9 and taking the bounding box of the 1-standard-deviation contour ellipsoid of the Gaussian (*i.e.*, $\lambda = 1$), we obtain the quadratic equation:

$$((\mathbf{f} \circ P_2^\top) \cdot P_2) c^2 - 2((\mathbf{f} \circ P_0^\top) \cdot P_2) c + (\mathbf{f} \circ P_0^\top) \cdot P_0 = 0, \quad (\text{D.2})$$

1309 where $\mathbf{f} = (1, 1, 1, -1)$ is a 1×4 row vector. We write this compactly as:

$$\mathcal{T}_{22}c^2 - 2\mathcal{T}_{02}c + \mathcal{T}_{00} = 0, \quad (\text{D.3})$$

where $\mathcal{T}$ is a 3×3 upper-triangular matrix with entries $\mathcal{T}_{ij} = (f \circ P_i^\top) \cdot P_j$.

Analogously, the bounds of ϕ are the roots $c = \tan \phi_{1,2}$ of:

1315 $\mathcal{T}_{22}c^2 - 2\mathcal{T}_{12}c + \mathcal{T}_{11} = 0. \quad (\text{D.4})$

Notably, for a camera-view configuration $[R_c \mid \mathbf{t}_c]$, the parameter matrix $\mathcal{T}$ is obtained directly and efficiently from the Gaussian’s view-space covariance $\Sigma_c = R_c R S S^\top R^\top R_c^\top$ and 3D mean $\boldsymbol{\mu}_c = R_c \boldsymbol{\mu} + \mathbf{t}_c$:

$$\mathcal{T}_{ij} = \lambda^2 \Sigma_c^{i,j} - (\mu_c \mu_c^\top)^{i,j}. \quad (\text{D.5})$$

1322 The angular bounds—specified by their tangent-space center and extent—then follow as:

$$\begin{aligned} & \left(\frac{\tan \theta_1 + \tan \theta_2}{2}, \frac{\tan \phi_1 + \tan \phi_2}{2} \right) = \left(\frac{\mathcal{T}_{02}}{\mathcal{T}_{22}}, \frac{\mathcal{T}_{12}}{\mathcal{T}_{22}} \right), \\ & \|\tan \theta_1 - \tan \theta_2\| = \frac{2\sqrt{\mathcal{T}_{02}^2 - \mathcal{T}_{22}\mathcal{T}_{00}}}{|\mathcal{T}_{22}|}, \\ & \|\tan \phi_1 - \tan \phi_2\| = \frac{2\sqrt{\mathcal{T}_{12}^2 - \mathcal{T}_{22}\mathcal{T}_{11}}}{|\mathcal{T}_{22}|}. \end{aligned} \tag{D.6}$$

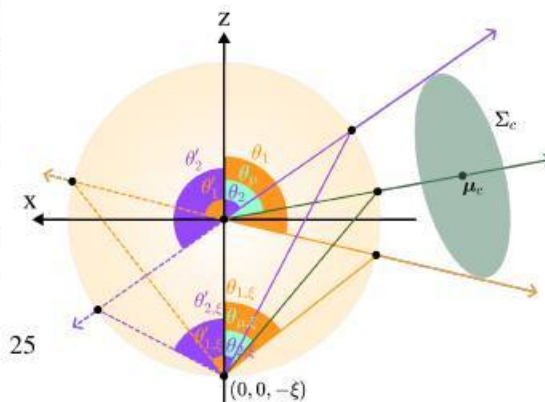
Whenever the discriminant of either quadratic is negative—signaling that the optical center lies inside the Gaussian ellipsoid, leaving the angular bounds undefined—we conservatively clamp the frustum angles to the camera’s maximum field of view (FoV).

Crucially, this result is computed directly from the true 3D view-space covariance, whereas EWA and UT must route through intermediate conic approximations in screen space. The PBF formulation therefore secures exactness and efficiency in a single construction, rather than trading one against the other.

1339 D.2 DETAILS IN COMPUTING CSF-PBF INTERSECTION

We uniformly sample spherical angles within the maximum field of view (FoV). The resulting x , y , and z coordinates—the ray direction vectors $\mathbf{d}$ —are projected onto a unit sphere and transformed into pixel space for inverse color interpolation (see Sec. E), then grouped by their corresponding CSFs for the subsequent Gaussian association.

For most pinhole cameras with limited FoV, the PBF angular bounds lie within $[-\frac{\pi}{2}, \frac{\pi}{2}]$,



over which $\tan$ is monotonically increasing; PBF–CSF intersections then reduce to a direct comparison of the $\tan$ values of their angular bounds. Under wide FoV, by contrast, the PBF angular bounds can fall outside the monotonic region of $\tan$ even when the incidence angle at the Gaussian center still lies within $[-\frac{\pi}{2}, \frac{\pi}{2}]$. A direct comparison of $\tan$ values then risks spurious PBF–CSF intersections—for instance, mistakenly pairing the bounds as $[\theta'_2, \theta_1]$ (see Fig. D.1).

To recover the correct intersections, we apply a mirror transformation to the tangent values of both the PBF angular bounds and the CSF partitions. Following the CMEI camera model (Mei & Rives 2007), we shift the optical center by a mirror parameter ξ , which gives the transformation:

$$\tan \theta_\xi = \frac{\tan \theta}{1 + \xi \frac{z}{\|z\|} \sqrt{1 + \tan^2 \theta}} = \frac{\sin \theta}{\cos \theta + \xi}, \quad (\text{D.7})$$

where θ_ξ is the mirror-transformed angle of θ on the xz -plane under the mirror parameter ξ ; we fix $\xi = 1$ throughout all experiments.

For a pair of tangent values $[\tan \theta_1, \tan \theta_2]$ from the PBF solver (Eq. D.6), this transformation produces four candidate mirror angles (Fig. D.1), whereas the Gaussian center corresponds to a single mirror-transformed angle $\theta_{\mu, \xi}$. Exploiting this asymmetry, we select the two candidates closest to $\theta_{\mu, \xi}$ as the effective PBF bounds $[\theta_{1, \xi}, \theta_{2, \xi}]$, and then compare them with the CSF partitions to obtain the correct intersections.

This construction guarantees robust, consistent angular comparisons even beyond the monotonic domain of $\tan$, and it does so at negligible cost. As the timing analysis reports (Tab. K.5), the entire preprocessing pipeline—mirror-angle PBF–CSF intersection together with the canonical transformation of all Gaussians—completes in under 0.13 ms in total, so that the approach remains efficient even for the thousands of Gaussians that populate wide-FoV scenes.

D.3 RELATION TO PRIOR WORKS

Substituting the perspective screen-space coordinates

$$u = \tan \theta, \quad v = \tan \phi$$

into our formulation reproduces exactly the expressions of 2DGS (Huang et al. 2024a) and HTGS (Hahlbohm et al. 2025), as one would expect: these are precisely the perspective specialization of our angular-domain formulation.

Concretely, 2DGS and HTGS begin from a screen-space or tangent-plane parameterization and therefore invoke assumptions valid only for perspective (pinhole) cameras—for instance, geometric identities that hold only when rays are implicitly tied to a planar, z -parallel image plane. Such assumptions do not carry over to fisheye or omnidirectional systems, where the corresponding screen-space surface is curved. Our derivation instead proceeds from a camera-agnostic angular parameterization in 3D, directly parameterizing Camera Sub-Frustums (CSFs) and Particle Bounding Frustums (PBFs). These weaker starting assumptions change the known quantities and constraints available during derivation, and with them the mathematical conditions under which closed-form bounds and culling guarantees remain valid for generic cameras.

We stress that although mathematics is a general tool, deriving these bounds under weaker assumptions—without recourse to screen-space structure—and validating them experimentally across both pinhole and wide-FoV camera families is itself an integral part of our contribution, not a mere formal restatement of prior results.

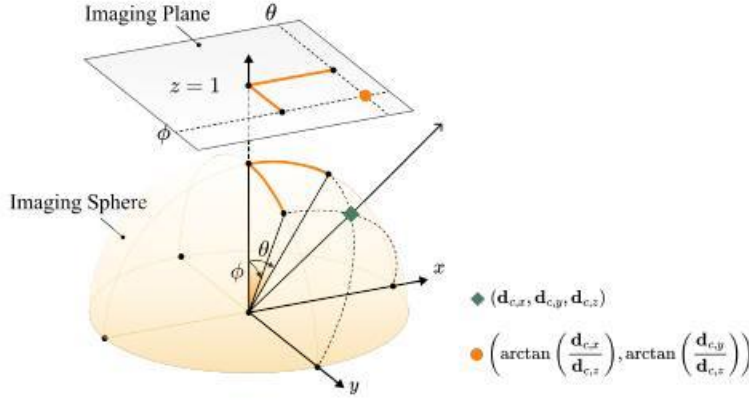


Figure E.1: **Spherical Angles Defining PBF and BEAP.** A view-space direction $\mathbf{d}_c$ is normalized onto the optical sphere (green spade) and then projected angularly onto the xz and yz planes; the orange point marks the resulting coordinate (θ, ϕ) on the unfolded BEAP imaging plane. The same pair of angles parameterizes both the PBF and the BEAP representation, which is what lets association and supervision share a single construction.

This difference in parameterization propagates through the entire downstream computational pipeline. Gaussian association, tiling/duplication, and sorting all hinge on how Gaussians map to tiles or bins, and an angular (CSF-based) domain induces a bucketing structure distinct from pinhole screen-space tiling. The distinction is consequential for large-FoV scenes: under generic cameras a Gaussian’s bound may lie partially “behind” the camera in screen coordinates (Fig. D.1), causing screen-space methods to emit mirrored or incorrect tangent-based bounds (dashed line in Fig. D.1). To handle these cases robustly, we introduce mirror parameters in the association step (Sec. D.2) and fold the Gaussian center into sorting and correction—components that perspective-only derivations never require and cannot supply.

E APPENDIX: BEAP IMAGE INVERSE INTERPOLATION

To prepare training data, we sample rays that comprehensively cover the native image FoV and feed them as input to our CUDA renderer. For each input ray we concurrently interpolate the ground-truth color, so that supervision is applied accurately and uniformly throughout training. Because evaluation is subsequently carried out in the native image space (Sec. G), this resampling supplies balanced supervision without biasing the reported metrics toward the BEAP domain.

Projecting the sampled rays back onto the observed images begins by transforming the incidence-angle projections θ and ϕ into normalized ray direction vectors $\mathbf{d} = (x, y, z)^\top$:

$$x = \frac{\sin \theta \cos \phi}{\sqrt{\sin^2 \theta \cos^2 \phi + \cos^2 \theta \sin^2 \phi + \cos^2 \theta \cos^2 \phi}}, \quad (\text{E.1})$$

$$y = \frac{\cos \theta \sin \phi}{\sqrt{\sin^2 \theta \cos^2 \phi + \cos^2 \theta \sin^2 \phi + \cos^2 \theta \cos^2 \phi}}, \quad (\text{E.2})$$

$$z = \frac{\cos \theta \cos \phi}{\sqrt{\sin^2 \theta \cos^2 \phi + \cos^2 \theta \sin^2 \phi + \cos^2 \theta \cos^2 \phi}}. \quad (\text{E.3})$$

We support several camera models for the subsequent projection of the normalized rays $(x, y, z)^\top$ into pixel space $(x_p, y_p, 1)^\top$, so that the same supervision pipeline accommodates both pinhole and fisheye inputs.

Pinhole Camera Model. For the pinhole model, $\frac{x}{z} = \tan \theta$ and $\frac{y}{z} = \tan \phi$, so that:

$$\begin{bmatrix} x_p \\ y_p \\ 1 \end{bmatrix} = \begin{bmatrix} f_x & 0 & c_x \\ 0 & f_y & c_y \\ 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} \tan \theta \\ \tan \phi \\ 1 \end{bmatrix}. \quad (\text{E.4})$$

KB Fisheye Model. The incidence angle φ is recovered from its projections:

$$r = \sqrt{\tan^2 \theta + \tan^2 \phi} \quad \text{and} \quad \varphi = \arctan(r). \quad (\text{E.5})$$

The fisheye-distorted angle then follows as:

$$\varphi_d = \varphi(1 + k_1\varphi^2 + k_2\varphi^4 + k_3\varphi^6 + k_4\varphi^8). \quad (\text{E.6})$$

The rays consequently project into pixel space, permitting inverse color interpolation from:

$$\begin{bmatrix} x_p \\ y_p \\ 1 \end{bmatrix} = \begin{bmatrix} \frac{f_x \varphi_d}{r} & 0 & c_x \\ 0 & \frac{f_y \varphi_d}{r} & c_y \\ 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} \tan \theta \\ \tan \phi \\ 1 \end{bmatrix}. \quad (\text{E.7})$$

F APPENDIX: HIGH-LEVEL ALGORITHMIC SUMMARY

We give a high-level overview of our volumetric rendering pipeline (Alg. 1 and Alg. 2). To render the differences from standard splatting immediately legible, we mirror the structure of 3DGS [Kerbl et al., 2023]: newly introduced or replaced modules are highlighted in green, and substantially modified modules in yellow. The layout makes clear that our contributions are localized to association and rendering, leaving the surrounding training loop intact.

ALGORITHM 1: GPU Software Asso. & Render.

Function 3DGEER ($\mu, \mathbf{s}, \mathbf{q}, c, \sigma, \mathbf{r}_c, V$):

```

  CullGaussian( $\mu, V$ )                                ▷ Frustum Culling
   $F \leftarrow \text{InitCSFs}(w, h, K)$                         ▷ Partition
   $\Sigma_c, \mu_c \leftarrow \text{ViewspaceGaussians}(\mathbf{s}, \mathbf{q}, V)$     ▷ Transform
   $F_{ID} \leftarrow \text{PBF\_Intersect}(\Sigma_c, \mu, F)$         ▷ Sub-frustum Asso.
   $W_{pca} \leftarrow \text{PCA\_Whitening}(\mathbf{s}, \mathbf{q})$                 ▷ Transform
   $\mathcal{G}_{ID}, F_K \leftarrow \text{DuplicateWithKeys}(W_{pca}, F_{ID})$     ▷ Indices (Gaussian) & Keys (FrustumID+Depth)
  SortByKeys( $\mathcal{G}_{ID}, F_K$ )                                ▷ Globally Depth Sort
   $R \leftarrow \text{IdentifyFrustumRanges}(F, F_K)$ 
   $I \leftarrow 0$                                           ▷ Init Canvas (w/ Rays)
  for all CSFs  $f \leftarrow I$  do
    for all Rays  $i \leftarrow f$  do
       $r \leftarrow \text{GetFrustumRange}(R, f)$ 
       $\mathbf{m}_u, \mathbf{d}_u \leftarrow \text{Cano\_Ray}(W_{pca}, \mu, i)$         ▷ Transform
       $D_{\mu, \Sigma} \leftarrow \text{Dist}(\mathbf{m}_u, \mathbf{d}_u)$             ▷ Mah-distance
       $I[i] \leftarrow \text{BlendInOrder}(i, \mathcal{G}_{ID}, r, F_K, D_{\mu, \Sigma}, c, \sigma)$ 
    end
  end
  return  $I$ 

```

G APPENDIX: ADDITIONAL IMPLEMENTATION DETAILS

Baselines. We compare 3DGEER against the leading volumetric particle rendering methods that target the projective approximation error of 3DGS [Kerbl et al., 2023]. For large-FoV fisheye inputs, the baselines are FisheyeGS [Liao et al., 2024], EVER [Mai et al., 2025], and 3DGUT [Wu et al., 2025]; under pinhole settings, we additionally include 3DGS and 3DGRT [Moënné-Loccoz et al., 2024].

ALGORITHM 2: 3DGEER Initialization and Training Framework

w, h : width and height of the training images

K, V : intrinsics and extrinsic meta info.

```
1512  $\mu \leftarrow$  SfM Points ▷ Init Gaussian Attributes
1513  $\mathbf{s}, \mathbf{q} \leftarrow$  InitAttributes() ▷ Position
1514  $c, \sigma \leftarrow$  InitCoefficients() ▷ Scaling, Rotation
1515 ▷ Color, Opacity
1516 while not converged do
1517    $\mathbf{r}_c, \hat{C} \leftarrow$  BEAP( $w, h, K$ ) ▷ View Rays / Colors
1518    $C \leftarrow$  3DGEER( $\mu, \mathbf{s}, \mathbf{q}, c, \sigma, \mathbf{r}_c, V$ ) ▷ Asso. & Render
1519    $\mathcal{L} \leftarrow$  Loss( $C, \hat{C}$ ) ▷ See Alg. 1
1520    $\mu, \mathbf{s}, \mathbf{q}, c, \sigma \leftarrow$  Adam( $\nabla \mathcal{L}$ ) ▷ Loss
1521   if IsRefinementIteration( $i$ ) then ▷ Backprop & Step
1522      $\mu, \mathbf{s}, \mathbf{q}, c, \sigma \leftarrow$  Optimize( $\mu, \mathbf{s}, \mathbf{q}, c, \sigma$ ) ▷ Kerbl et al. (2023)
1523   end
1524 end
```

Implementation Details. To keep the comparison fair, we align our implementation with the original 3DGS in initialization and hyperparameters on the ScanNet++ and MipNeRF360 datasets, and on ZipNeRF we adopt the training configurations of EVER and SMERF (Duckworth et al., 2024) for all candidate methods, so that no method benefits from a bespoke schedule. As is characteristic of ray-based frameworks, our method does not directly access `mean2D` gradients to steer densification or pruning; it instead optimizes using gradients of the view-space `mean3D`. Unless otherwise noted, all models are trained under consistent supervision strategies and architectural settings, and all are run for the same 30k-iteration budget. For the fisheye datasets (ScanNet++ and ZipNeRF), we precompute bijective grid mappings (Guo et al., 2025) and reproject every rendered output to the native KB-Fisheye image space (Kannala & Brandt, 2006) before evaluation, thereby removing any representation bias between methods. ScanNet++ and MipNeRF360 experiments use full-resolution images; for ZipNeRF, given its high frame count and very large fisheye resolution, we adopt the maximum resolution (see Tab. 3) at which all baselines run reliably on a local desktop.

Central and peripheral regions in ScanNet++ evaluation. The rays assigned to the central region follow the range

$$\theta \in [-\frac{w}{2f_x}, \frac{w}{2f_x}], \phi \in [-\frac{h}{2f_y}, \frac{h}{2f_y}],$$

and the peripheral region comprises the rays sampled in the remaining angular ranges. Here w, h denote the width and height of the image and f_x, f_y the focal lengths of the original KB camera model. This partition isolates the region where distortion, and hence projective error, is most severe.

Cross-camera generalization evaluation on ZipNeRF. ZipNeRF provides two sets of data: fish-eye raw images and their corresponding undistorted frames, the latter obtained by applying distortion correction and cropping the central regions. Although the frames are aligned on a per-frame basis, camera poses and point clouds were reconstructed independently with COLMAP (Schönberger & Frahm, 2016) for each set. Because COLMAP’s global optimization refines the two trajectories independently, a single rigid transformation cannot align the two pose sets exactly and may itself introduce alignment error. To secure a reliable cross-camera evaluation free of structure-from-motion (SfM) artifacts, we therefore reuse the same set of camera poses at test time as during training, varying only the intrinsic parameters while holding the extrinsic poses fixed. This design keeps the comparison fair by preventing pose inconsistencies introduced during preprocessing from contaminating the reported cross-camera results.

H APPENDIX: EWA SPLATTING PRELIMINARIES

From camera coordinates to ray coordinates. Following the coordinate definitions of EWA Splatting (Zwicker et al. 2001), the mapping from camera (view) space points $\mathbf{t}$ to ray space points $\mathbf{x}$ under the projective transformation of a pinhole camera, i.e., $\tau : \mathbf{t} \rightarrow \mathbf{x}$, is:

$$\mathbf{x} = \begin{bmatrix} x_0 \\ x_1 \\ x_2 \end{bmatrix} = \tau(\mathbf{t}) = \begin{bmatrix} t_0/t_2 \\ t_1/t_2 \\ \|(t_0, t_1, t_2)^\top\| \end{bmatrix}, \quad (\text{H.1})$$

$$\mathbf{t} = \begin{bmatrix} t_0 \\ t_1 \\ t_2 \end{bmatrix} = \tau^{-1}(\mathbf{x}) = \begin{bmatrix} x_0/l \cdot x_2 \\ x_1/l \cdot x_2 \\ 1/l \cdot x_2 \end{bmatrix}, \quad (\text{H.2})$$

$$, \text{ where } l = \|(x_0, x_1, 1)^\top\|. \quad (\text{H.3})$$

These mappings are non-affine, and computing the ray-space integral under them therefore demands either computationally expensive solutions (Westover 1990; Mao et al. 1995) or the locally approximated ones (Zwicker et al. 2001; Kerbl et al. 2023) that our method sets out to avoid.

Local affine approximation. The EWA splatting of Zwicker et al. (2001) introduces a local affine approximation, denoted τ_k , that preserves the linear effects and keeps the k -indexed reconstruction kernels Gaussian in 2D screen space. Defined by the first two terms of the Taylor expansion of τ at the camera-space point $\mathbf{t}_k$, it estimates the ray-space point $\mathbf{x}$ in the neighborhood of $\mathbf{x}_k$ as:

$$\mathbf{x} = \tau(\mathbf{t}) = \mathbf{x}_k + \mathbf{J}_\tau^{\mathbf{t}_k}(\mathbf{t} - \mathbf{t}_k), \quad (\text{H.4})$$

where $\mathbf{J}_\tau^{\mathbf{t}_k}$ is the Jacobian matrix formed from the partial derivatives of τ at $\mathbf{t}_k$. For the k -indexed 3D Gaussian $\mathcal{G}_{\Sigma, \mu}$ in a given camera coordinate, the EWA local affine transformation expresses the approximated density function as a 3D Gaussian $\mathcal{G}_{\Sigma_k, \mathbf{x}_k}$ in the ray coordinate. For $\forall \mathbf{x}$ around $\mathbf{x}_k$, we have:

$$\mathcal{G}_{\Sigma_k, \mathbf{x}_k}(\mathbf{x}) = \frac{1}{(2\pi)^{\frac{3}{2}} |\Sigma_k|^{\frac{1}{2}}} \exp\left(-\frac{1}{2}(\mathbf{x} - \mathbf{x}_k)^\top \Sigma_k^{-1}(\mathbf{x} - \mathbf{x}_k)\right), \quad (\text{H.5})$$

$$\text{where } \Sigma_k = \mathbf{J}_\tau^{\mathbf{t}_k} \Sigma (\mathbf{J}_\tau^{\mathbf{t}_k})^\top. \quad (\text{H.6})$$

Integrating a 3D Gaussian $\mathcal{G}_{\Sigma_k, \mathbf{x}_k}^{(3)}$ along one coordinate axis yields a 2D Gaussian $\mathcal{G}_{\hat{\Sigma}_k, \hat{\mathbf{x}}_k}^{(2)}$, so that:

$$\int_{\mathbb{R}} \mathcal{G}_{\Sigma_k, \mathbf{x}_k}^{(3)}(\mathbf{x}) dx_2 = \mathcal{G}_{\hat{\Sigma}_k, \hat{\mathbf{x}}_k}^{(2)}(\hat{\mathbf{x}}), \quad (\text{H.7})$$

where $\hat{\mathbf{x}} = \begin{bmatrix} x_0 \\ x_1 \end{bmatrix}$ and the mean $\hat{\mathbf{x}}_k = \begin{bmatrix} x_{k,0} \\ x_{k,1} \end{bmatrix}$, and the 2×2 variance matrix $\hat{\Sigma}_k$ is read off the 3×3 matrix Σ_k by dropping its third row and column.

Splatting approximation error. Although the density function in ray space cannot be expected to remain Gaussian once a general projective transformation is applied to the original view space, the exact density is recoverable by substitution through Eq. H.2

$$\begin{aligned} \mathcal{G}_{\Sigma, \mu}(\mathbf{t}) &= \mathcal{G}_{\Sigma, \mu}(\tau^{-1}(\mathbf{x})) \\ &= \frac{1}{(2\pi)^{\frac{3}{2}} |\Sigma_k|^{\frac{1}{2}}} \exp\left(-\frac{1}{2}(\tau^{-1}(\mathbf{x}) - \mathbf{t}_k)^\top \Sigma_k^{-1}(\tau^{-1}(\mathbf{x}) - \mathbf{t}_k)\right), \end{aligned} \quad (\text{H.8})$$

so that the approximation error can be quantified by Monte Carlo integration of:

$$\mathcal{E}_k = \int_{\mathbb{R}} |\mathcal{G}_{\Sigma, \mu}(\tau^{-1}(\mathbf{x})) - \mathcal{G}_{\Sigma_k, \mathbf{x}_k}(\mathbf{x})| dx_2. \quad (\text{H.9})$$

Here x_2 denotes the dimension along the ray direction in the ray coordinate. This residual is exactly the error that our projective-exact formulation removes.



Figure I.1: **Qualitative Results on the Stump Scene (Mip-NeRF360).** On this pinhole novel-view-synthesis benchmark (*Left*), 3DGEER outperforms both 3DGS-based (Gaussian Splatting) and 3DPRT-based (Particle Ray Tracing) methods while retaining high runtime efficiency. The most pronounced differences appear within the colored boxes.



Figure J.1: **Qualitative Results on the Aria Dataset.** Our method recovers sharper floor textures in the indoor scene and clearer distant tree detail in the outdoor scene than the baseline.

I APPENDIX: ADVANTAGE OVER PRIOR RAY-TRACING METHODS

Pure ray-tracing (e.g., 3DGRIT) can in principle be projective-exact, yet in practice the BVH that 3DGRIT employs for intersection tests relies on an inscribed proxy icosahedron for speed—a proxy that is tight at the vertices but whose face centers lie roughly 20% inward, a fixed property of the icosahedron. Such proxy shapes degrade intersection accuracy, corrupt the accumulated gradients that drive density control, and can ultimately yield incorrect geometry (e.g., the missing leaves in Fig. I.1). Intersection tests for far, tiny, and cluttered Gaussians are further limited by floating-point precision, which introduces noisy rendering (for instance, the native rendering in the leftmost block of Fig. 15 of the 3DGRIT paper shows visible speckles across the tree and background). Indicatively, even 3DGUT with its UT approximation outperforms 3DGRIT on MipNeRF360. To validate this directly, we render the same pretrained MipNeRF360-stump model through both 3DGRIT’s ray-tracing engine and our 3DGEER pipeline and compare on PSNR $\uparrow$ / SSIM $\uparrow$ / LPIPS $\downarrow$: 3DGRIT attains 25.79 / 0.745 / 0.264, whereas ours attains 26.83 / 0.774 / 0.246—a gain of +1.04 / +0.029 / −0.018 under an otherwise identical Gaussian field. By combining a projective-exact and efficient pipeline with BEAP color supervision, 3DGEER thus achieves both markedly faster rendering and higher accuracy (Tab. 4).

J APPENDIX: RESULTS ON EGOCENTRIC DATASET

Recent egocentric devices typically feature wide FoVs, yet existing pipelines struggle to maintain reconstruction quality at full resolution or when restricted to cropped views. The difficulty is compounded by low-light imaging conditions, where the limited light intake of small sensors and aperture-constrained lenses aggravates the degradation.

We evaluate on the Aria egocentric dataset ($\sim 110^\circ$ circular FoV). Our method attains an average PSNR of 33.5 dB, consistently surpassing FisheyeGS (32.8 dB) across all metrics (Tab. J.1). Qualitatively, our reconstructions preserve sharper detail in the peripheral regions of the indoor scene (Fig. J.1 *Left*) and recover distant structures such as trees in the outdoor scene (Fig. J.1 *Right*). Tellingly, the +0.7 dB gain falls squarely between those observed on pinhole datasets (MipNeRF360: +0.3 dB; ZipNeRF-cross: +2.0 dB) and on the most severely distorted wide-FoV datasets (ScanNet++/ZipNeRF at 180° diagonal FoV: +0.9–4.8 dB); the improvement scales with distortion, exactly as Aria’s moderate FoV would predict, corroborating that 3DGEER’s advantage is driven by the elimination of projective error.

Table J.1: **Quantitative Comparison on Aria.** Higher PSNR/SSIM and lower LPIPS are better; 3DGEER leads FisheyeGS on every metric for both the indoor and the outdoor scene.

Method	Scene		PSNR $\uparrow$	SSIM $\uparrow$	LPIPS $\downarrow$
3DGEER (Ours)	livingroom	(indoor)	36.398	0.933	0.226
	steakhouse	(outdoor)	30.662	0.881	0.246
FisheyeGS	livingroom	(indoor)	35.686	0.927	0.231
	steakhouse	(outdoor)	29.989	0.869	0.261

K APPENDIX: ADDITIONAL MATERIAL AND RESULTS

K.1 ADDITIONAL MATERIAL.

Code, video, and an HTML page are provided in the Supplementary Material.

K.2 ADDITIONAL RESULTS.



Figure K.1: **Out-of-Distribution (OOD) Region in Fig. 6** 3DGS collapses to black owing to the numerical singularity $\tan 90^\circ \rightarrow \infty$, whereas our method renders robustly out to 180° and successfully reconstructs the green chair on which splat-based methods (e.g., FisheyeGS) fail. Consistently, in the zoom-in of Fig. 6 our method also preserves structures such as the corridor and fridge, the table and door, and the window and cabinet.



Figure K.2: **Extreme-FoV Generalization.** (Left) Our method versus 3DGS under extreme-FoV pinhole (170°) views, both trained only on undistorted central regions. (Right) A zoom-out of Fig. 6 spanning the complete circular 180° FoV, in which our method generalizes robustly to the out-of-distribution periphery.

Table K.1: **MipNeRF360 Quantitative Results by Scene.** Per-scene breakdown of the aggregate results reported in Tab. 4

Method	Metric	Bicycle	Bonsai	Counter	Garden	Kitchen	Stump	Flowers	Room	Treehill
Ours	PSNR $\uparrow$	25.63	32.23	29.05	27.60	31.78	26.83	21.83	32.09	22.83
	SSIM $\uparrow$	0.774	0.946	0.910	0.860	0.931	0.774	0.613	0.946	0.637
	LPIPS $\downarrow$	0.223	0.175	0.195	0.136	0.120	0.246	0.340	0.108	0.344

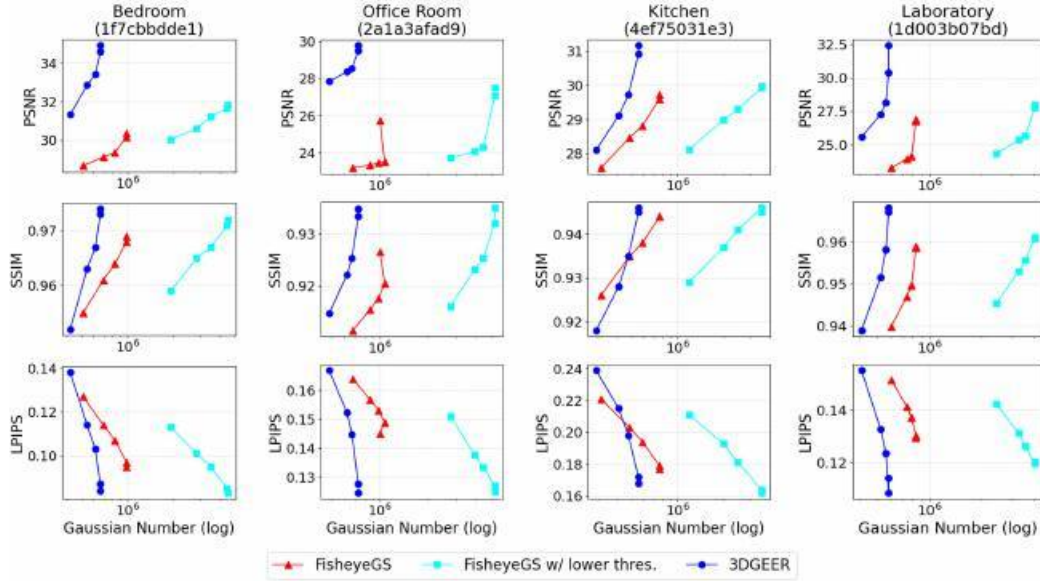


Figure K.3: **PSNR $\uparrow$, SSIM $\uparrow$, and LPIPS $\downarrow$ Trends When Scaling Up Gaussian Counts.** Across all three metrics, brute-force scaling of the Gaussian budget fails to close the gap in projective exactness, corroborating the PSNR trend of Fig. 5

Table K.2: **ScanNet++ Quantitative Results by Scene (Full FoV).** Per-scene breakdown of the aggregate results reported in Tab. 2; boldface marks the best value in each column.

Scene	EVER			FisheyeGS			3DGUT			3DGEER (Ours)		
	PSNR	SSIM	LPIPS	PSNR	SSIM	LPIPS	PSNR	SSIM	LPIPS	PSNR	SSIM	LPIPS
Storage	28.24	0.916	0.164	24.74	0.932	0.146	29.26	0.934	0.147	29.18	0.940	0.141
Office Room	27.80	0.899	0.169	25.75	0.927	0.145	29.37	0.928	0.145	29.78	0.935	0.125
Bedroom	32.60	0.948	0.131	31.95	0.970	0.095	34.65	0.967	0.112	34.94	0.974	0.084
Kitchen	29.77	0.924	0.213	29.72	0.944	0.177	30.78	0.941	0.196	31.17	0.946	0.168
Laboratory	28.96	0.936	0.156	26.91	0.959	0.129	29.12	0.948	0.151	32.43	0.968	0.114



Figure K.4: **Qualitative Results on the ScanNet++ Dataset.** The most pronounced differences appear in the zoomed-in colored boxes, particularly across structural detail and artifact regions. Our method delivers superior peripheral reconstruction relative to FisheyeGS, exhibits fewer artifacts and sharper detail than EVER, and produces crisper object boundaries and textures—wires, keyboards, and wooden floors among them—than 3DGUT.

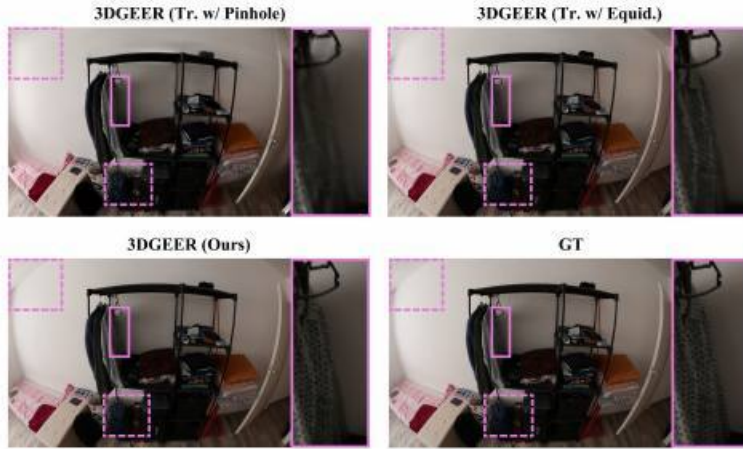


Figure K.5: **Impact of BEAP Supervision.** A visual comparison on ScanNet++ shows that BEAP’s balanced angular sampling yields smoother gradients and preserves geometric and texture detail more faithfully than either pinhole or equidistant sampling (Tab. K.8 reports the full quantitative statistics).

Table K.3: **Quantitative Results on ScanNet++: Trends in PSNR $\uparrow$, SSIM $\uparrow$, and LPIPS $\downarrow$ When Scaling Up Gaussian Counts.** Scenes appear in the order Bedroom, Office Room, Kitchen, and Laboratory. Even at Gaussian counts several-fold larger, the scaled-up splat-based variant does not overtake 3DGEER, which uses far fewer Gaussians throughout.

Iter	FisheyeGS				FisheyeGS w/ Low Thres.				3DGEER			
	PSNR $\uparrow$	SSIM $\uparrow$	LPIPS $\downarrow$	Gau $\downarrow$	PSNR $\uparrow$	SSIM $\uparrow$	LPIPS $\downarrow$	Gau $\downarrow$	PSNR $\uparrow$	SSIM $\uparrow$	LPIPS $\downarrow$	Gau $\downarrow$
4k	28.67	0.955	0.127	512K	30.03	0.959	0.113	1914K	31.33	0.952	0.138	422K
7k	29.12	0.961	0.114	695K	30.61	0.965	0.101	2829K	32.87	0.963	0.114	543K
10k	29.36	0.964	0.107	827K	31.21	0.967	0.095	3489K	33.42	0.967	0.103	616K
20k	30.17	0.968	0.097	985K	31.68	0.971	0.085	4467K	34.59	0.973	0.087	664K
30k	30.36	0.969	0.095	985K	31.84	0.972	0.083	4535K	34.94	0.974	0.084	664K

Iter	FisheyeGS				FisheyeGS w/ Low Thres.				3DGEER			
	PSNR $\uparrow$	SSIM $\uparrow$	LPIPS $\downarrow$	Gau $\downarrow$	PSNR $\uparrow$	SSIM $\uparrow$	LPIPS $\downarrow$	Gau $\downarrow$	PSNR $\uparrow$	SSIM $\uparrow$	LPIPS $\downarrow$	Gau $\downarrow$
4k	23.19	0.911	0.164	677K	23.72	0.916	0.151	2778K	27.84	0.915	0.167	480K
7k	23.33	0.916	0.157	867K	24.06	0.923	0.138	3936K	28.37	0.922	0.152	621K
10k	23.45	0.918	0.153	983K	24.31	0.925	0.133	4481K	28.54	0.925	0.145	668K
20k	23.51	0.921	0.149	1072K	27.07	0.932	0.127	5272K	29.49	0.933	0.128	732K
30k	25.75	0.927	0.145	1001K	27.50	0.935	0.125	5272K	29.78	0.935	0.125	732K

Iter	FisheyeGS				FisheyeGS w/ Low Thres.				3DGEER			
	PSNR $\uparrow$	SSIM $\uparrow$	LPIPS $\downarrow$	Gau $\downarrow$	PSNR $\uparrow$	SSIM $\uparrow$	LPIPS $\downarrow$	Gau $\downarrow$	PSNR $\uparrow$	SSIM $\uparrow$	LPIPS $\downarrow$	Gau $\downarrow$
4k	27.58	0.926	0.221	314K	28.12	0.929	0.211	1191K	28.11	0.918	0.239	292K
7k	28.47	0.935	0.203	485K	28.99	0.937	0.193	1997K	29.11	0.928	0.215	410K
10k	28.82	0.938	0.194	588K	29.29	0.941	0.181	2491K	29.73	0.935	0.198	476K
20k	29.60	0.944	0.179	760K	29.93	0.946	0.164	3563K	30.91	0.945	0.172	552K
30k	29.72	0.944	0.177	760K	29.97	0.945	0.162	3563K	31.17	0.946	0.168	552K

Iter	FisheyeGS				FisheyeGS w/ Low Thres.				3DGEER			
	PSNR $\uparrow$	SSIM $\uparrow$	LPIPS $\downarrow$	Gau $\downarrow$	PSNR $\uparrow$	SSIM $\uparrow$	LPIPS $\downarrow$	Gau $\downarrow$	PSNR $\uparrow$	SSIM $\uparrow$	LPIPS $\downarrow$	Gau $\downarrow$
4k	23.27	0.940	0.152	602K	24.32	0.945	0.142	2429K	25.59	0.939	0.155	406K
7k	23.94	0.947	0.141	740K	25.36	0.953	0.131	3258K	27.27	0.952	0.133	524K
10k	24.12	0.950	0.137	788K	25.65	0.956	0.126	3569K	28.14	0.958	0.124	562K
20k	26.76	0.959	0.130	838K	27.77	0.961	0.120	4035K	30.39	0.967	0.108	582K
30k	26.91	0.959	0.129	838K	27.97	0.961	0.120	4035K	32.43	0.968	0.114	582K

Table K.4: **Runtime Analysis vs. Splatting-Based Methods.** We compare 3DGEER against efficient splatting pipelines on MipNeRF360 and ScanNet++, reporting both stage-wise and total runtime. PBF’s tight bounds curtail duplication and sorting, so total time remains on par with—and on ScanNet++ below—the splatting baselines despite the higher per-ray rendering cost.

Dataset	Method	Avg. Timings (ms) $\downarrow$				
		Prep.	Dup.	Sort	Render	Total
MipNeRF360	3DGS	0.32	0.27	0.65	1.40	2.92
	3DGEER (Ours)	0.37	0.17	0.27	2.10	3.06
ScanNet++	FisheyeGS	0.10	0.63	1.45	2.33	4.70
	3DGEER (Ours)	0.13	0.26	0.59	2.89	3.98

Table K.5: **Runtime Analysis of the Association Method on ScanNet++**. We compare PBF-based association against the splatting-based EWA and UT schemes—each with the SnugBox accelerator—all integrated into the identical 3DGEER pipeline for a controlled comparison. PBF is the fastest overall, reaching 251 FPS even against the SnugBox-accelerated alternatives.

3DGEER Asso.	Prep.	Avg. Timings (ms) ↓				Avg. FPS ↑
		Dup.	Sort	Render	Total	
EWA	0.13	1.17	3.27	12.69	17.84	56
+ SnugBox	0.13	0.60	1.42	6.22	8.68	115
UT	0.13	0.71	1.94	7.97	11.16	90
+ SnugBox	0.13	0.28	0.64	3.16	4.38	228
PBF (Ours)	0.13	0.26	0.59	2.89	3.98	251

Table K.6: **Impact of PBF-CSF Association**. Evaluation uses the artifact-sensitive LPIPS metric. To probe cross-model robustness, each variant is additionally tested on Gaussian fields trained under the other association strategies. We also report the rendering-time memory cost, the mean and standard deviation of Gaussians associated per tile, and the gap Δ LPIPS between in-method and cross-method evaluation as a measure of consistency and generalization. PBF associates 3–5 $\times$ fewer Gaussians per tile and exhibits by far the smallest Δ LPIPS, indicating that it generalizes cleanly to fields trained by other methods.

Asso. Method	Gaus. ↓ per Tile		Mem. ↓ (GB)	LPIPS ↓	Cross-LPIPS ↓	Δ LPIPS ↓ (1e-2)
	mean	std				
EWA	2203	1447	2.2	0.1250	0.1321	0.71
UT	1377	1058	1.4	0.1249	0.1302	0.53
PBF (Ours)	475	427	0.63	0.1245	0.1251	0.06

Table K.7: **Training-Time Transmittance Function Replacement**. Per-scene breakdown of the aggregate results reported in Tab. 5. Note that our projective-exact formulation attains the best quality while using markedly fewer Gaussians than either splat-based transmittance.

Scene	3DGS Splats + PBF				FisheyeGS Splats + PBF				3DGEER (Ours)			
	PSNR	SSIM	LPIPS	Gaus. Num (k)	PSNR	SSIM	LPIPS	Gaus. Num (k)	PSNR	SSIM	LPIPS	Gaus. Num (k)
Storage	19.27	0.859	0.173	1293	24.75	0.933	0.150	781	29.18	0.940	0.141	429
Office Room	20.89	0.848	0.182	1527	25.89	0.928	0.150	1119	29.78	0.935	0.125	732
Bedroom	27.39	0.923	0.119	1542	31.96	0.970	0.096	1079	34.94	0.974	0.084	664
Kitchen	25.15	0.890	0.239	1241	29.87	0.949	0.182	801	31.17	0.946	0.168	552
Laboratory	21.60	0.869	0.173	1379	27.03	0.962	0.131	823	32.43	0.968	0.114	582

Table K.8: **Impact of the Supervision Space**. Per-scene breakdown of the aggregate results reported in Tab. 6 with the projective-exact integral and PBF association held fixed so that only the supervision sampling varies. BEAP supervision is best or tied-best across scenes.

Supervision Space Scene	3DGEER w/o BEAP Central Perspective			3DGEER w/o BEAP Perspective			3DGEER w/o BEAP Equidistant			3DGEER (Ours) BEAP		
	PSNR	SSIM	LPIPS	PSNR	SSIM	LPIPS	PSNR	SSIM	LPIPS	PSNR	SSIM	LPIPS
Storage	28.42	0.931	0.143	19.74	0.834	0.302	29.03	0.938	0.142	29.18	0.940	0.141
Office Room	28.90	0.927	0.133	21.12	0.825	0.291	29.71	0.932	0.126	29.78	0.935	0.125
Bedroom	33.86	0.965	0.090	24.38	0.882	0.240	34.60	0.973	0.096	34.94	0.974	0.084
Kitchen	30.65	0.941	0.172	23.23	0.889	0.338	31.23	0.941	0.177	31.17	0.946	0.168
Laboratory	27.35	0.951	0.118	17.08	0.834	0.329	30.65	0.956	0.135	32.43	0.968	0.114

Table K.9: ZipNeRF Quantitative Results by Scene. Scenes appear in the order Alameda, Berlin, London, and NYC; the aggregate results are reported in Tab. 3. The per-scene figures confirm that 3DGEER’s cross-camera advantage is consistent rather than driven by any single scene.

Train Data	1/8 FE		1/8 FE		1/8 FE		1/4 PH		1/4 PH		1/4 PH	
Test Data	1/8 FE	1/4 PH	1/8 FE	1/4 PH	1/8 FE	1/4 PH	1/8 FE	1/4 PH	1/8 FE	1/4 PH	1/8 FE	1/4 PH
Method	PSNR $\uparrow$		SSIM $\uparrow$		LPIPS $\downarrow$		PSNR $\uparrow$		SSIM $\uparrow$		LPIPS $\downarrow$	
FisheyeGS	20.74	23.64	0.820	0.802	0.223	0.277	17.39	23.74	0.736	0.839	0.265	0.240
EVER	22.46	23.06	0.852	0.777	0.144	0.274	20.06	22.83	0.789	0.794	0.213	0.250
3DGUT	21.78	22.39	0.841	0.718	0.193	0.368	17.03	22.58	0.588	0.765	0.329	0.311
Ours	24.60	25.00	0.857	0.839	0.154	0.241	21.49	25.21	0.792	0.807	0.232	0.277

Train Data	1/8 FE		1/8 FE		1/8 FE		1/4 PH		1/4 PH		1/4 PH	
Test Data	1/8 FE	1/4 PH	1/8 FE	1/4 PH	1/8 FE	1/4 PH	1/8 FE	1/4 PH	1/8 FE	1/4 PH	1/8 FE	1/4 PH
Method	PSNR $\uparrow$		SSIM $\uparrow$		LPIPS $\downarrow$		PSNR $\uparrow$		SSIM $\uparrow$		LPIPS $\downarrow$	
FisheyeGS	24.25	28.25	0.895	0.924	0.206	0.209	20.08	27.86	0.848	0.929	0.233	0.194
EVER	25.26	26.97	0.912	0.907	0.155	0.221	23.18	27.72	0.877	0.913	0.196	0.212
3DGUT	25.37	27.02	0.904	0.876	0.197	0.288	18.57	27.43	0.730	0.898	0.297	0.249
Ours	25.51	29.15	0.916	0.925	0.166	0.194	23.58	29.51	0.878	0.915	0.219	0.228

Train Data	1/8 FE		1/8 FE		1/8 FE		1/4 PH		1/4 PH		1/4 PH	
Test Data	1/8 FE	1/4 PH	1/8 FE	1/4 PH	1/8 FE	1/4 PH	1/8 FE	1/4 PH	1/8 FE	1/4 PH	1/8 FE	1/4 PH
Method	PSNR $\uparrow$		SSIM $\uparrow$		LPIPS $\downarrow$		PSNR $\uparrow$		SSIM $\uparrow$		LPIPS $\downarrow$	
FisheyeGS	23.57	26.65	0.829	0.861	0.253	0.254	20.83	27.33	0.768	0.883	0.288	0.229
EVER	24.59	26.83	0.823	0.859	0.218	0.223	23.14	26.98	0.793	0.860	0.268	0.235
3DGUT	25.61	26.25	0.854	0.803	0.221	0.336	19.82	26.41	0.669	0.833	0.313	0.291
Ours	26.60	27.84	0.895	0.882	0.142	0.230	23.95	27.15	0.840	0.846	0.217	0.289

Train Data	1/8 FE		1/8 FE		1/8 FE		1/4 PH		1/4 PH		1/4 PH	
Test Data	1/8 FE	1/4 PH	1/8 FE	1/4 PH	1/8 FE	1/4 PH	1/8 FE	1/4 PH	1/8 FE	1/4 PH	1/8 FE	1/4 PH
Method	PSNR $\uparrow$		SSIM $\uparrow$		LPIPS $\downarrow$		PSNR $\uparrow$		SSIM $\uparrow$		LPIPS $\downarrow$	
FisheyeGS	24.15	27.21	0.890	0.885	0.162	0.215	19.41	27.50	0.811	0.907	0.203	0.183
EVER	28.39	27.71	0.931	0.860	0.095	0.228	25.22	28.27	0.879	0.883	0.154	0.192
3DGUT	26.31	26.69	0.917	0.817	0.121	0.302	19.00	27.42	0.659	0.866	0.263	0.228
Ours	28.26	28.49	0.922	0.906	0.097	0.190	24.52	28.55	0.874	0.885	0.168	0.220